Pinku Clinic ▪ ระบบจองคิวคลินิก

เอกสารอธิบายการทำงานของโปรแกรม

วิชา CPE361 · Python 3 + sqlite3 + Tkinter + tkcalendar · ฐานข้อมูล clinic.db · ปรับปรุงล่าสุด 24 ตุลาคม 2569

ผู้นำเสนอ	ส่วนที่รับผิดชอบในสไลด์	หัวข้อในเอกสารนี้
A1	เปิดเรื่อง เครื่องมือ (รวม tkcalendar) ภาพรวมลำดับการทำงาน แถบหัว หน้าต่างแจ้งเตือน และปิดท้าย	หัวข้อ 1–2 และ 1.3–1.4 ในขั้นที่ 1
B1	ER Diagram 6 ตาราง กฎของระบบ ตัวอย่าง SQL การเชื่อมต่อ transaction และ SQL สรุปตัวเลขวันนี้	ขั้นที่ 1 และหัวข้อ 8–9
C1	ลงทะเบียนผู้ป่วย ปฏิทินวันเกิด ประวัติการรักษา และจองคิว + Demo	ขั้นที่ 2–3
D1	ค้นหาคิว ยกเลิกคิว บันทึกว่าตรวจแล้วพร้อมผลวินิจฉัย + Demo	ขั้นที่ 4–5

เอกสารนี้ไล่ตาม **ลำดับการทำงาน 5 ขั้น** เดียวกับในสไลด์ ทุกขั้นตอบ 5 คำถามเหมือนกัน: ผู้ใช้กดอะไร » โปรแกรมเรียกฟังก์ชันไหนในไฟล์ไหน » รัน SQL อะไร » ข้อมูลในตารางเปลี่ยนอย่างไร (มีตัวอย่างก่อนและหลัง) » แล้วไปต่อส่วนไหน

ท้ายเอกสารเป็น **คำถามที่อาจารย์น่าจะถาม 32 ข้อ** แยกตามส่วนของผู้นำเสนอ แต่ **ทุกคนต้องอ่านทั้งหมด** เพราะอาจารย์ถามใครก็ได้

> **ตัวอย่างข้อมูลในเอกสารนี้**
>
> ตัวเลขทุกตัวมาจากการรันโปรแกรมจริงบนเครื่อง Windows (Python 3.15, SQLite 3.53.4) หลัง python create_db.py เมื่อ 24 ตุลาคม 2569 ข้อมูลชุดตั้งต้น: แผนก 5, แพทย์ 10, ผู้ป่วย 15, ช่องเวลา 50, คิว 30, ผลการรักษา 15
>
> ภาพหน้าจอทุกภาพถ่ายจากโปรแกรมจริงตามสคริปต์ Demo วันที่ 24 ตุลาคม 2569 ข้อมูลตัวอย่างคิดวันที่จากวันที่รัน create_db.py จึงได้ตัวเลขชุดเดียวกันทุกครั้ง แต่ "วันนี้" และ "พรุ่งนี้" เลื่อนตามวันที่รัน (ในเอกสารนี้ วันนี้ = 2026-10-24, พรุ่งนี้ = 2026-10-25)
>
> เรื่องตัวอย่างที่ใช้ทั้งเล่ม: ลงทะเบียน **ชไมพร ใจกว้าง** » จองคิวกับ **พญ.กมลวรรณ ทองดี พรุ่งนี้ 10:00** ได้ **คิวที่ 31** » ค้นหาเจอ » ยกเลิกคิว » บันทึกว่า **คิวที่ 7** ตรวจแล้ว พร้อมผลวินิจฉัย

1. ภาพรวมระบบ

ผู้ใช้คือ **เจ้าหน้าที่คลินิก 1 คน** ไม่มีหน้า Login ผู้ป่วยไม่ได้ใช้โปรแกรมเอง แต่โทรมาหรือมาที่เคาน์เตอร์ แล้วเจ้าหน้าที่ทำรายการให้ โปรแกรมเป็นหน้าต่างเดียว ด้านบนเป็นแถบหัวแสดงชื่อระบบและตัวเลขสรุปคิววันนี้ ด้านล่างแบ่งเป็น 3 แท็บ

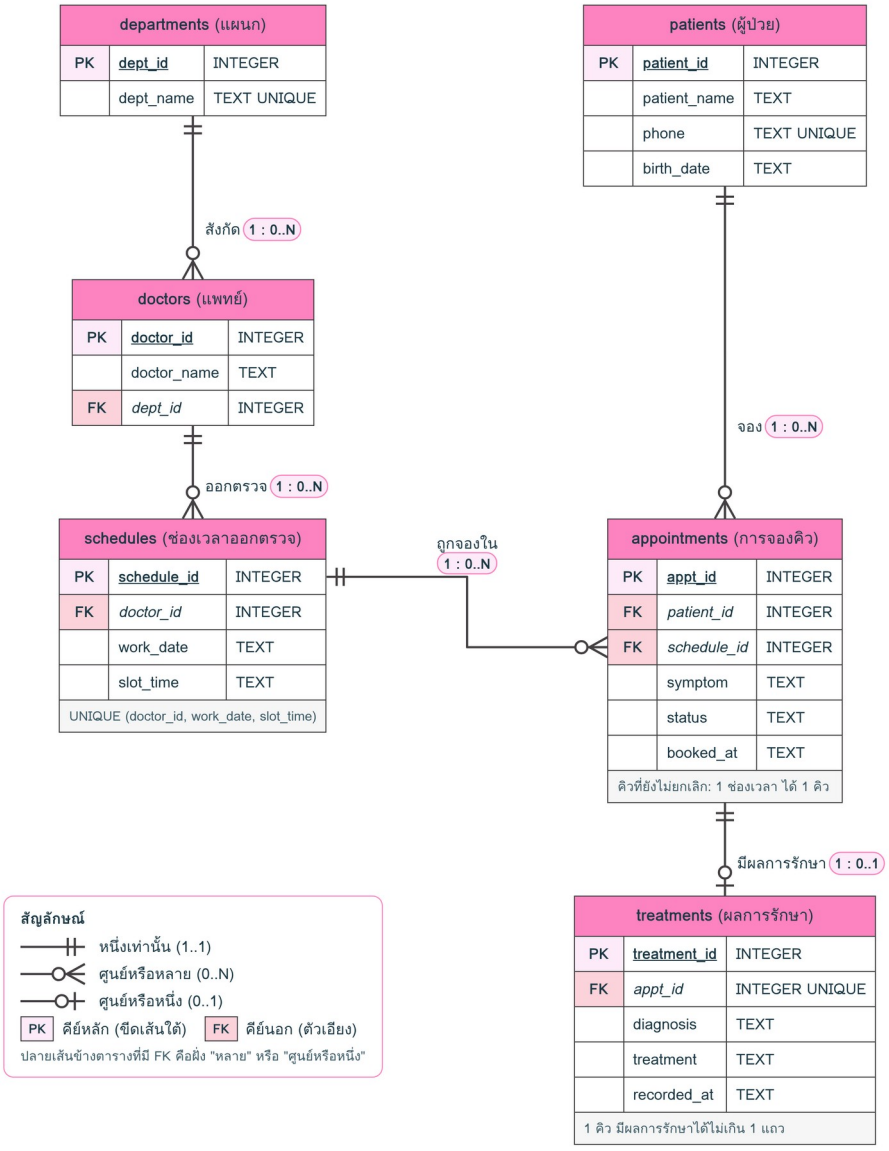
ไฟล์	หน้าที่	ขั้นที่ใช้
main.py	หน้าต่างหลัก ตรวจว่ามีฐานข้อมูลและ tkcalendar สร้างแถบหัวและ 3 แท็บ	1
database.py	เชื่อมต่อ clinic.db + เปิด foreign key, รัน SQL ด้วย ?, transaction, สรุปคิววันนี้, แปลง error เป็นภาษาไทย	ทุกขั้น
theme.py	ชื่อโปรแกรม (Pinku Clinic) สีชมพู และฟอนต์ Tahoma ของทุก widget	ทุกขั้น
header_bar.py	แถบหัว: ชื่อระบบ + ตัวเลข คิวทั้งหมด / ตรวจแล้ว / รอตรวจ / ยกเลิกของวันนี้	ทุกขั้น
dialogs.py	หน้าต่างแจ้งเตือน 4 แบบที่เข้ากับธีม ใช้แทน messagebox	ทุกขั้น
patient_tab.py	แท็บ "ผู้ป่วย" ลงทะเบียน แก้ไข เลือกวันเกิดจากปฏิทิน และดูประวัติการรักษา	2
booking_tab.py	แท็บ "จองคิว" ค้นผู้ป่วย เลือกแผนก แพทย์ วันที่ ช่องเวลาที่ว่าง แล้วบันทึก	3
search_tab.py	แท็บ "ค้นหาและจัดการคิว" ค้นหา ยกเลิก บันทึกว่าตรวจแล้วพร้อมฟอร์มผลวินิจฉัย	4–5
create_db.py + clinic_db.sql	ลบ clinic.db เดิม แล้วสร้าง 6 ตารางและข้อมูลตัวอย่างใหม่ (รีเซ็ตก่อน Demo)	ก่อนขั้นที่ 1

6 ตารางในฐานข้อมูล

ตาราง	เก็บอะไร	คีย์และข้อบังคับสำคัญ	แถวตั้งต้น
departments	แผนก	dept_name ห้ามซ้ำ	5
doctors	แพทย์ และแผนกที่สังกัด	dept_id เป็น FK » departments	10
patients	ผู้ป่วย	phone ห้ามซ้ำ ต้องเป็น 0 + เลข 9 หลัก, birth_date ต้องเป็นวันที่จริง	15
schedules	ช่องเวลาออกตรวจ ช่องละ 30 นาที	UNIQUE (doctor_id, work_date, slot_time)	50
appointments	การจองคิว	status = จองแล้ว / ตรวจแล้ว / ยกเลิก, partial unique index: 1 ช่องมีคิวที่ยังไม่ยกเลิกได้ 1 คิว	30
treatments	ผลการรักษาของคิวที่ตรวจแล้ว	appt_id เป็น FK » appointments และ UNIQUE (1 คิว ต่อ 0..1 ผลการรักษา), diagnosis ห้ามว่าง	15

สถานะคิวในข้อมูลตั้งต้น: ตรวจแล้ว 15 · จองแล้ว 12 · ยกเลิก 3 และคิวที่ตรวจแล้วทุกคิวมีผลการรักษา (create_db.py ตรวจแล้วได้ "คิวที่ตรวจแล้วแต่ยังไม่มีผลการรักษา: 0 คิว")

ER Diagram ของ Pinku Clinic · ระบบจองคิวคลินิก (Crow's Foot)



ER Diagram แบบ Crow's Foot (รายละเอียดอยู่ในเอกสารขั้นที่ 1 docs/CPE361_step1_database_design.pdf)

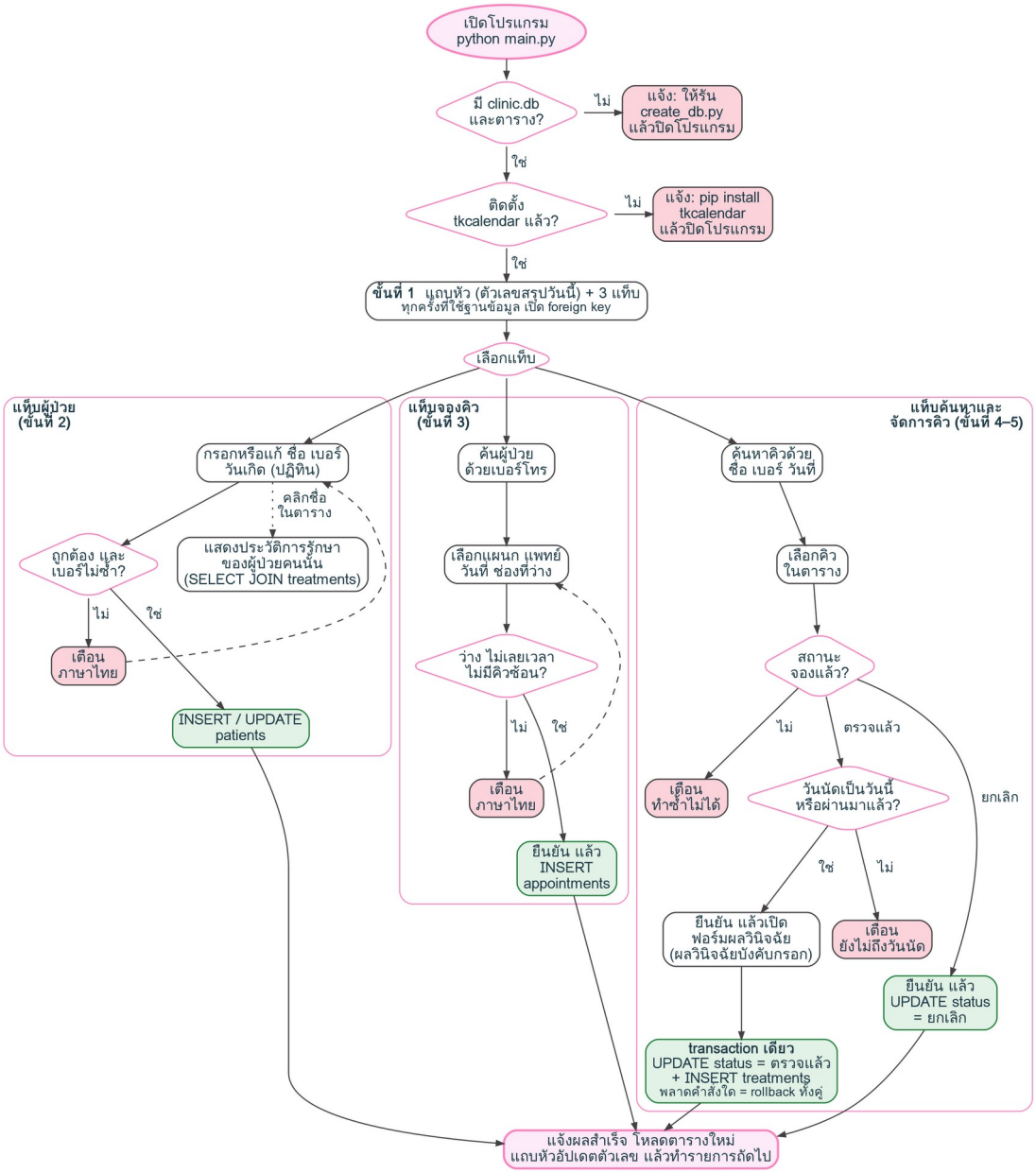
ฟังก์ชันใน database.py ที่ทุกแท็บใช้

ฟังก์ชัน	ใช้กับ	คืนค่า
get_connection()	ถูกเรียกโดยทุกฟังก์ชันด้านล่าง	การเชื่อมต่อที่สั่ง PRAGMA foreign_keys = ON แล้ว
database_ready()	ตอนเปิดโปรแกรม	True ถ้ามี clinic.db และตาราง appointments
fetch_all(sql, params)	SELECT หลายแถว	list ของ tuple
fetch_one(sql, params)	SELECT แถวเดียว	tuple หรือ None
execute(sql, params)	UPDATE / DELETE	จำนวนแถวที่ถูกแก้ (rowcount)
insert(sql, params)	INSERT	รหัสของแถวใหม่ (lastrowid)
execute_transaction(commands)	บันทึกว่าตรวจแล้ว (UPDATE + INSERT)	True ถ้าทุกคำสั่งสำเร็จ, False ถ้ามีคำสั่งที่แก้ 0 แถว (rollback แล้ว)

ฟังก์ชัน	ใช้กับ	คืนค่า
get_today_summary()	แถบหัวโปรแกรม	(วันที่, ทั้งหมด, ตรวจแล้ว, รอตรวจ, ยกเลิก) ของวันนี้
error_to_thai(error)	ตอนฐานข้อมูลปฏิเสธคำสั่ง	ข้อความภาษาไทยสำหรับหน้าต่างแจ้งเตือน

2. แผนภาพลำดับการทำงานของระบบ

อ่านจากบนลงล่าง: เปิดโปรแกรม » ตรวจฐานข้อมูลและ tkcalendar » เลือกแท็บ แต่ละแท็บมีจุดตรวจ (รูปเพชร) ก่อนเขียนข้อมูลเสมอ ฝั่งบันทึกว่าตรวจแล้ว: ยืนยัน » ฟอร์มผลวินิจฉัย » transaction



สีเขียว = คำสั่งที่เขียนลงฐานข้อมูล · สีชมพูแดง = แจ้งเตือนภาษาไทยแล้วกลับไปแก้ (เส้นประ) · ถ้าฐานข้อมูลปฏิเสธคำสั่ง (UNIQUE / FOREIGN KEY / CHECK / NOT NULL) จะผ่าน `error_to_thai()` ให้เป็นข้อความภาษาไทยก่อนแสดง (หัวข้อ 8)

ขั้นที่ 1 เปิดโปรแกรม » เชื่อมต่อ clinic.db และเปิด foreign key

ส่วนของ B1 (การเชื่อมต่อ) และ A1 (แถบหัว หน้าต่างแจ้งเตือน) · ไฟล์ main.py database.py header_bar.py dialogs.py

หัวข้อ	สรุป
ผู้ใช้ทำอะไร	พิมพ์ `python main.py` ใน Terminal (ต้องติดตั้ง tkcalendar และรัน `python create_db.py` มาก่อนอย่างน้อย 1 ครั้ง)
ฟังก์ชันที่ทำงาน	`main()` » `setup_style()` » `database.database_ready()` » ตรวจ tkcalendar » `header_bar.create_bar()` » `create_tab()` ของ 3 แท็บ » `root.mainloop()`
SQL ที่รัน	`PRAGMA foreign_keys = ON`, SELECT หาตาราง appointments และ SQL สรุปตัวเลขวันนี้
ข้อมูลเปลี่ยนไหม	ไม่เปลี่ยน ขั้นนี้อ่านอย่างเดียว
ไปต่อ	หน้าต่างเปิดที่แท็บ "ผู้ป่วย" » ขั้นที่ 2 (ผู้ป่วยเคยลงทะเบียนแล้ว ไปขั้นที่ 3 ได้เลย)

1.1 โปรแกรมทำอะไรทีละขั้น

1. `main.py` » `main()` สร้างหน้าต่าง `tk.Tk()` ชื่อ "Pinku Clinic - ระบบจองคิวคลินิก" ขนาด 1100×720 แล้ว `theme.setup_style()` ตั้งสีชมพูและฟอนต์ Tahoma ทั้งโปรแกรม
2. เรียก `database.database_ready()` ตรวจว่ามีไฟล์ clinic.db ก่อน แล้วค่อยหาตาราง appointments (ต้องตรวจไฟล์ก่อน connect เพราะถ้าไม่มีไฟล์ `sqlite3.connect()` จะสร้างไฟล์เปล่าให้เอง) ไม่พร้อม » แจ้ง "ไม่พบฐานข้อมูล" แล้วปิดโปรแกรม
3. ตรวจว่า import tkcalendar ได้ ถ้าไม่ได้ » แจ้ง "ยังไม่ได้ติดตั้ง tkcalendar" พร้อมคำสั่ง `python -m pip install tkcalendar` แล้วปิดโปรแกรม
4. `header_bar.create_bar()` สร้างแถบหัว แล้วเรียก `refresh()` ใส่ตัวเลขของวันนี้
5. สร้าง 3 แท็บด้วย `patient_tab.create_tab()`, `booking_tab.create_tab()`, `search_tab.create_tab()` แต่ละแท็บโหลดข้อมูลแรกเข้ามาเอง
6. ผูกเหตุการณ์สลับแท็บกับ `on_tab_changed()` ทุกครั้งที่สลับ จะเรียก `header_bar.refresh()` และ `refresh()` ของแท็บนั้น ให้ข้อมูลเป็นปัจจุบัน แล้วเริ่ม `root.mainloop()`

1.2 การเชื่อมต่อฐานข้อมูล (database.py)

โปรแกรม **เปิดการเชื่อมต่อใหม่ทุกครั้งที่ใช้ฐานข้อมูล** แล้วปิดทันที ทุกการเชื่อมต่อผ่าน `get_connection()` ซึ่งสั่ง `PRAGMA foreign_keys = ON` ก่อนเสมอ เพราะ SQLite ปิดการตรวจ foreign key เป็นค่าเริ่มต้น และค่านี้ไม่ได้เก็บในไฟล์

database.py (ย่อ)

```
def get_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("PRAGMA foreign_keys = ON")
    return conn

def execute(sql, params=()):
    conn = get_connection()
    cursor = conn.cursor()
    try:
        cursor.execute(sql, params)      # ค่าส่งแยกจากคำสั่ง แทน ? ตามลำดับ
        conn.commit()
        count = cursor.rowcount
    finally:
        conn.close()                     # ปิดเสมอ แม้เกิด error ไฟล์จะไม่ค้างล็อก
    return count
```

1.3 แถบหัวโปรแกรมและตัวเลขสรุปวันนี้ (header_bar.py)

ส่วนของ A1 (หน้าตา) และ B1 (SQL)



แถบหัวหลังรัน create_db.py (24 ตุลาคม 2569): คิวทั้งหมด 7 · ตรวจแล้ว 0 · รอตรวจ 6 · ยกเลิก 1

แถบหัวพื้นชมพู (PRIMARY #FD82C0) ตัวหนังสือสีเข้ม #1E3A47 แสดง "Pinku Clinic" กับ "ระบบจองคิวคลินิก" ทางซ้าย และกล่องตัวเลข 4 กล่องทางขวา ตัวเลขมาจาก database.get_today_summary() ซึ่งเป็น SQL คำสั่งเดียว:

get_today_summary()

```
SELECT date('now', 'localtime'),
       COUNT(*),                                                -- คิวทั้งหมดของวันนี้
       IFNULL(SUM(CASE WHEN a.status = 'ตรวจแล้ว' THEN 1 ELSE 0 END), 0), -- ตรวจแล้ว
       IFNULL(SUM(CASE WHEN a.status = 'จองแล้ว' THEN 1 ELSE 0 END), 0),  -- รอตรวจ
       IFNULL(SUM(CASE WHEN a.status = 'ยกเลิก' THEN 1 ELSE 0 END), 0)    -- ยกเลิก
FROM appointments a
JOIN schedules s ON a.schedule_id = s.schedule_id      -- วันที่ของคิวอยู่ที่ schedules
WHERE s.work_date = date('now', 'localtime')
```

เมื่อไรตัวเลขเปลี่ยน	เรียกจากที่ไหน	ตัวอย่างจากการรันจริง
เปิดโปรแกรม และทุกครั้งที่สลับแท็บ	main.on_tab_changed()	7 / 0 / 6 / 1 หลังรีเซ็ต
จองคิวสำเร็จ	booking_tab.save_booking()	จองพรุ่งนี้ ตัวเลขวันนี้จึงคงเดิม 7 / 0 / 6 / 1
ยกเลิกคิวสำเร็จ	search_tab.cancel_appointment()	ยกเลิกคิว 31 ของพรุ่งนี้ ตัวเลขวันนี้คงเดิม 7 / 0 / 6 / 1
บันทึกว่าตรวจแล้วสำเร็จ	search_tab.mark_done()	บันทึกคิว 7 ของวันนี้ » คิวทั้งหมด 7 · ตรวจแล้ว 1 · รอตรวจ 5 · ยกเลิก 1

COUNT(*) นับทุกคิวของวันนี้รวมคิวที่ยกเลิก ส่วน SUM(CASE WHEN ... THEN 1 ELSE 0 END) นับเฉพาะแถวที่ตรงสถานะ จึงได้ 4 ตัวเลขในแถวเดียว (7 = 0 + 6 + 1) ถ้าวันนั้นไม่มีคิวเลย SUM จะได้ NULL จึงครอบด้วย IFNULL(..., 0)

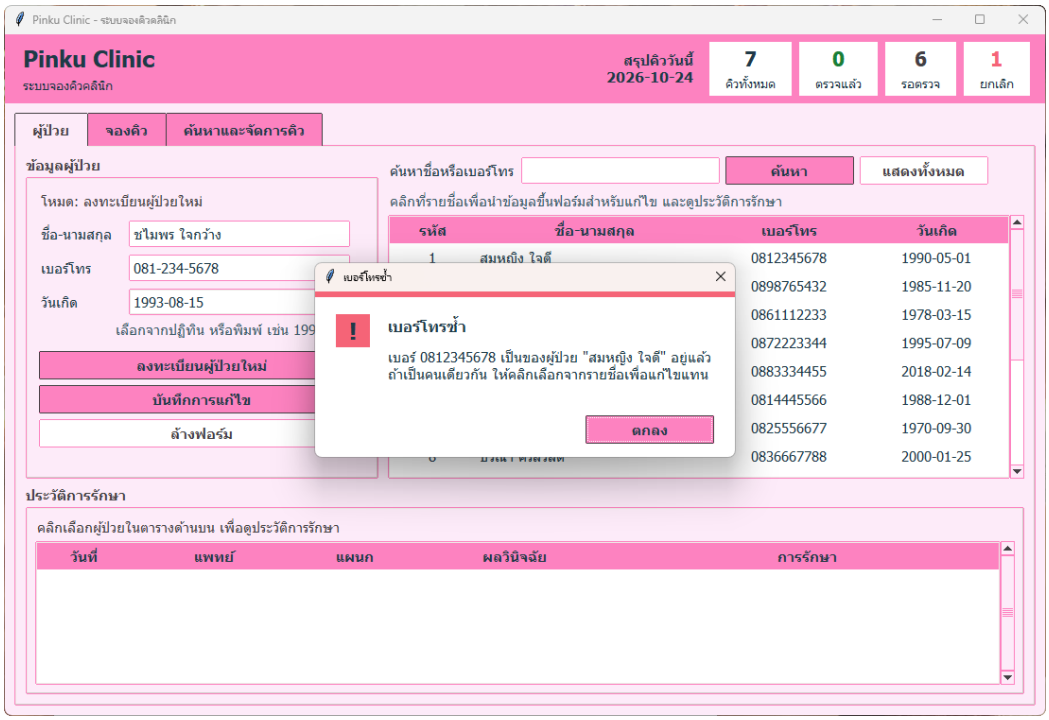
1.4 หน้าต่างแจ้งเตือนแบบใหม่ (dialogs.py)

ส่วนของ A1

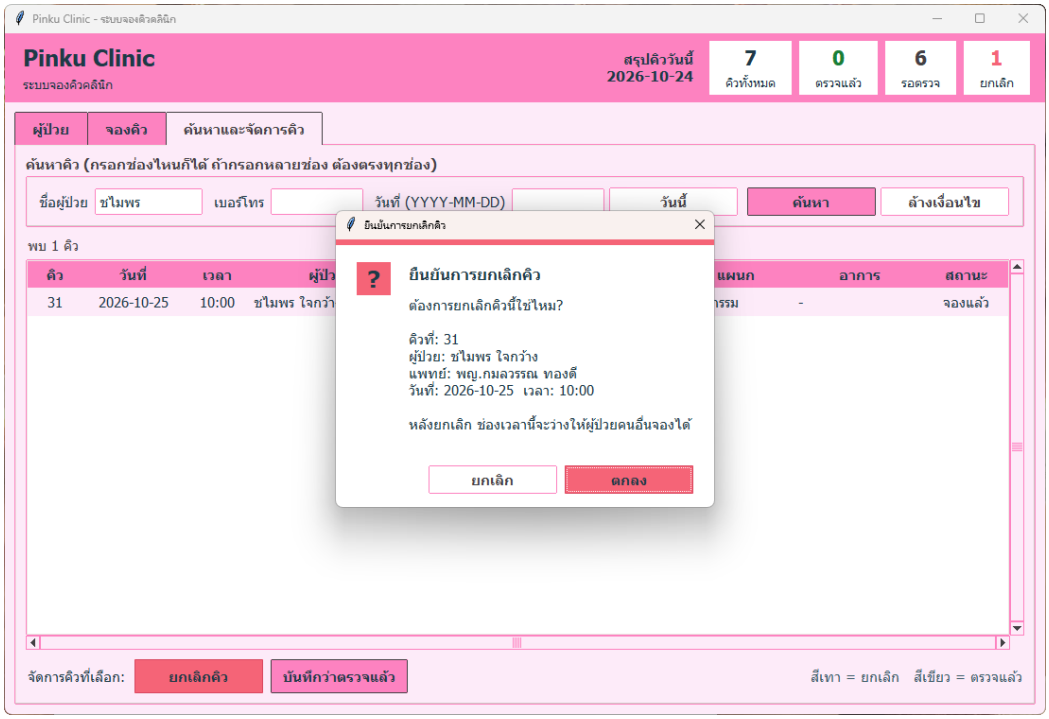
ทุกข้อความในโปรแกรมใช้ dialogs.py แทน messagebox ของ tkinter เพื่อให้สีและฟอนต์เข้ากับธีม และวางตรงกลางหน้าต่างหลักเสมอ

ฟังก์ชัน	แถบสีและสัญลักษณ์	ปุ่ม	ใช้เมื่อ
show_info()	ชมพู PRIMARY · i	ตกลง	บันทึกสำเร็จ เช่น "ลงทะเบียนสำเร็จ"
show_warning()	ชมพูแดง ACCENT · !	ตกลง	กรอกไม่ครบ เบอร์ซ้ำ คิวซ้อน ยังไม่ถึงวันนัด
show_error()	ชมพูแดง ACCENT · ×	ตกลง	ฐานข้อมูลปฏิเสธคำสั่ง (ข้อความจาก error_to_thai)
ask_yes_no()	ชมพู · ? (ถ้า icon="warning" ใช้สีเน้นและปุ่มตกลงสีเน้น)	ตกลง / ยกเลิก	ยืนยันการจอง ยืนยันบันทึกว่าตรวจแล้ว และยืนยันยกเลิกคิว (สีเน้น)

- **ล็อกหน้าต่างอื่น** ด้วย grab_set() ต้องปิดกล่องก่อนถึงจะกดหน้าต่างหลักได้ และ wait_window() รอคำตอบ
- **คีย์ลัด** Enter = ตกลง, Esc หรือกดกากบาท = ยกเลิก
- **ซ้อนกันได้** เช่น เตือน "กรุณากรอกผลวินิจฉัยก่อนบันทึก" ขึ้นบนฟอร์มผลวินิจฉัย ปิดแล้วการล็อกกลับไปที่ฟอร์มเดิม
- **ตัวหนังสือบนพื้นชมพูเป็นสีเข้ม #1E3A47** อ่านชัดบนโปรเจกเตอร์ ส่วนตอนชี้หรือกดปุ่ม พื้นเปลี่ยนเป็นสีเข้ม ตัวหนังสือจึงเปลี่ยนเป็นสีขาว



ตัวอย่าง show_warning(): ลงทะเบียนด้วยเบอร์ที่มีคนใช้แล้ว (ภาพจากโปรแกรมจริง)



ตัวอย่าง ask_yes_no(icon="warning"): ยืนยันการยกเลิกคิว แถบและปุ่มตกลงใช้สีเน้น

1.5 ข้อมูลที่โหลดตอนเปิดแท็บครั้งแรก (อ่านอย่างเดียว)

แท็บ	ฟังก์ชัน	ผลที่เห็น (ข้อมูลชุดตั้งต้น)
แถบหัว	header_bar.refresh()	คิวทั้งหมด 7 · ตรวจแล้ว 0 · รอตรวจ 6 · ยกเลิก 1
ผู้ป่วย	refresh()	รายชื่อผู้ป่วย 15 คน และกล่องประวัติการรักษาที่ว่างอยู่
จองคิว	load_departments()	แผนก 5 แผนกใน Combobox
ค้นหาและจัดการคิว	load_results()	ช่องวันที่ตั้งเป็นวันนี้ » "พบ 7 คิว"

ขั้นที่ 2 ลงทะเบียน แก้ไขข้อมูลผู้ป่วย และดูประวัติการรักษา

ส่วนของ C1 · ไฟล์ patient_tab.py

หัวข้อ	สรุป
ผู้ใช้ทำอะไร	กรอกชื่อ-นามสกุล เบอร์โทร วันเกิด (เลือกจากปฏิทินหรือพิมพ์) แล้วกด "ลงทะเบียนผู้ป่วยใหม่" หรือคลิกชื่อในตาราง แก้ แล้วกด "บันทึกการแก้ไข" การคลิกชื่อยังแสดงประวัติการรักษาด้วย
ฟังก์ชันที่ทำงาน	add_patient() / on_select() + update_patient() » read_form() » check_form() » database.insert() / database.execute() และ load_history()
SQL ที่รัน	SELECT เช็กเบอร์ซ้ำ » INSERT INTO patients หรือ UPDATE patients · SELECT ประวัติจาก treatments
ข้อมูลเปลี่ยน	patients เพิ่ม 1 แถว (ลงทะเบียน) หรือแก้ 1 แถว (แก้ไข) · การดูประวัติไม่เปลี่ยนข้อมูล
ไปต่อ	ขั้นที่ 3 ไปแท็บ "จองคิว" แล้วค้นผู้ป่วยด้วยเบอร์โทร

2.1 ลงทะเบียนผู้ป่วยใหม่

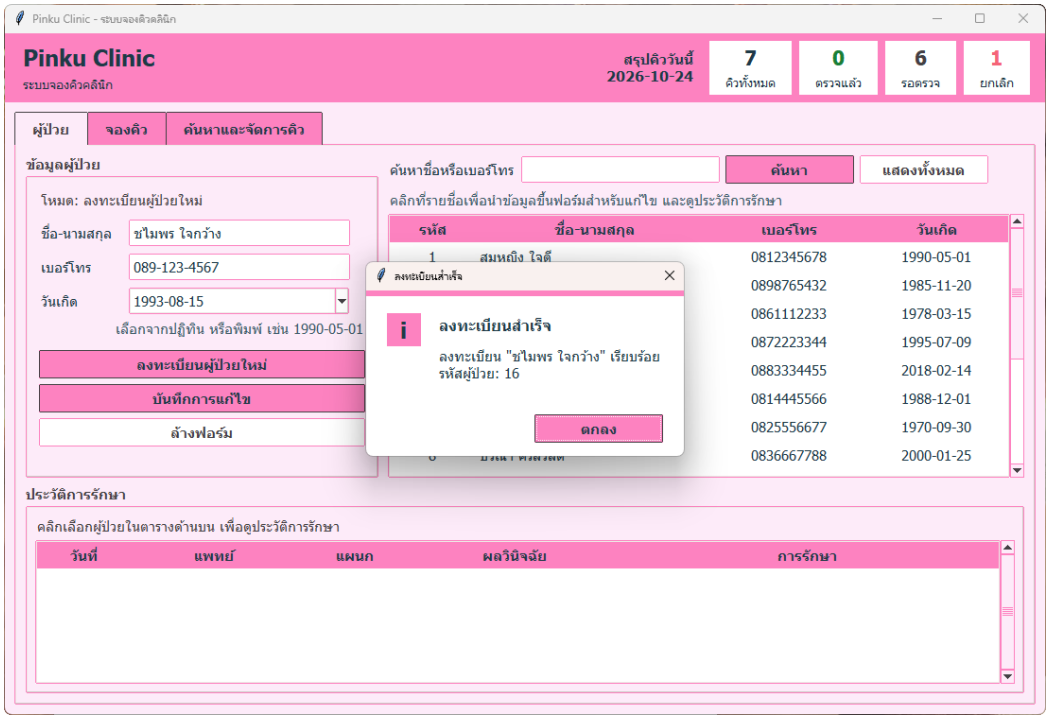
ตัวอย่าง: กรอก **ชไมพร ใจกว้าง**, เบอร์ **089-123-4567**, วันเกิด **1993-08-15** แล้วกดปุ่ม "ลงทะเบียนผู้ป่วยใหม่" (ถ้ากรอกเบอร์ 081-234-5678 ก่อน จะเจอกล่อง "เบอร์โทรซ้ำ" ในหัวข้อ 1.4)

1. add_patient() เรียก read_form() อ่านค่า ตัดช่องว่างหัวท้าย และตัด - กับช่องว่างออกจากเบอร์ (089-123-4567 » 0891234567)
2. check_form() ตรวจตามตารางด้านล่าง ไม่ผ่านข้อไหน » dialogs.show_warning() เตือนภาษาไทย แล้วหยุด
3. เช็กเบอร์ซ้ำด้วย SELECT ถ้าซ้ำ บอกด้วยว่าเป็นเบอร์ของใคร
4. database.insert() รัน INSERT ได้รหัสผู้ป่วยใหม่กลับมา » แจ้ง "ลงทะเบียนสำเร็จ ... รหัสผู้ป่วย: 16" » clear_form() » refresh() โหลดตารางใหม่

สิ่งที่ตรวจ (check_form)	ข้อความเตือน
มีช่องว่าง	กรุณากรอกชื่อ-นามสกุล เบอร์โทร และวันเกิดให้ครบ
เบอร์ไม่ใช่เลข 0–9 ครบ 10 หลักขึ้นต้นด้วย 0 (is_phone() ไม่ใช้ isdigit() เพราะรับเลขไทย ๐–๙ ด้วย)	เบอร์โทรต้องเป็นตัวเลข 10 หลัก และขึ้นต้นด้วย 0 เช่น 0812345678
วันเกิดไม่ใช่วันที่จริงแบบ YYYY-MM-DD (is_valid_date())	วันเกิดไม่ถูกต้อง กรุณากรอกแบบ YYYY-MM-DD เป็นปี ค.ศ. เช่น 1990-05-01
วันเกิดอยู่ในอนาคต (มักเกิดจากกรอกปี พ.ศ.)	วันเกิดต้องไม่เป็นวันในอนาคต ถ้ากรอกเป็นปี พ.ศ. ให้ลบ 543 เป็นปี ค.ศ. ก่อน
ปีเกิดก่อน ค.ศ. 1900	ปีเกิดต้องไม่ต่ำกว่า ค.ศ. 1900
เบอร์ซ้ำกับผู้ป่วยคนอื่น	เบอร์ ... เป็นของผู้ป่วย "..." อยู่แล้ว

SQL ที่รัน

```
SELECT patient_name FROM patients WHERE phone = ?;                    -- เช็กเบอร์ซ้ำ
INSERT INTO patients (patient_name, phone, birth_date) VALUES (?, ?, ?);   -- ค่า 3 ตัวส่งแทน ?
```



กล่อง "ลงทะเบียนสำเร็จ" ได้รหัสผู้ป่วย 16 (ภาพจากโปรแกรมจริง)

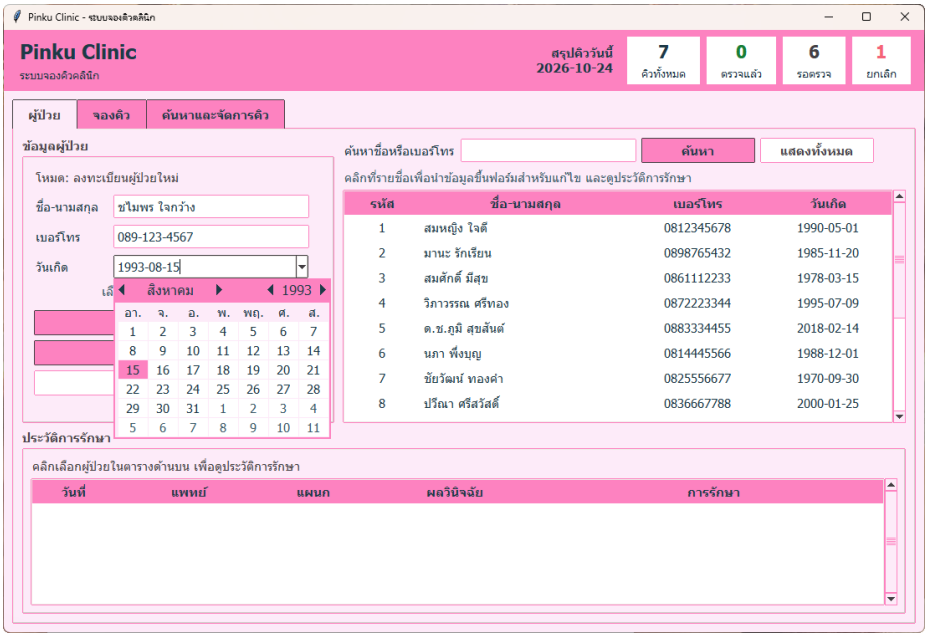
ตาราง patients ก่อนและหลัง

patient_id	patient_name	phone	birth_date
14	พงศกร แสงทอง	0932223344	1975-02-28
15	ลลิตา มณีรัตน์	0943334455	2002-07-07
16 (ใหม่)	**ชไมพร ใจกว้าง**	**0891234567**	**1993-08-15**

patient_id = 16 ได้จาก AUTOINCREMENT ไม่ต้องกรอกเอง insert() คืนค่านี้มาแสดงในกล่อง "ลงทะเบียนสำเร็จ" เบอร์ถูกเก็บแบบไม่มีขีด

2.2 ปฏิทินวันเกิด (tkcalendar)

ช่องวันเกิดเป็น DateEntry จากไลบรารี **tkcalendar** ซึ่งไม่ได้มากับ Python ต้องติดตั้งเพิ่ม 1 ครั้งด้วย python -m pip install tkcalendar (ติดตั้ง babel ให้อัตโนมัติ) กดลูกศรขวาของช่องจะเปิดปฏิทิน หรือพิมพ์วันที่เองก็ได้



ปฏิทินวันเกิดเปิดอยู่ (ภาพจากโปรแกรมจริง 24 ตุลาคม 2569)

ตั้งค่าใน patient_tab.py	ผล
date_pattern='yyyy-mm-dd'	เลือกวันจากปฏิทินแล้วช่องได้รูปแบบ YYYY-MM-DD ตรงกับ CHECK ในตาราง patients
locale='th_TH'	ชื่อเดือนและชื่อวันเป็นภาษาไทย แต่ปียังเป็น ค.ศ. (กันปัญหาปี พ.ศ.)
maxdate=date.today()	เลือกวันในอนาคตจากปฏิทินไม่ได้ (ลูกศรเลื่อนไปเดือนหน้าถูกปิดไว้)
firstweekday='sunday', showweeknumbers=False	สัปดาห์เริ่มวันอาทิตย์ ไม่แสดงเลขสัปดาห์
สีหัวปฏิทินและวันที่เลือก	พื้นชมพู PRIMARY ตัวหนังสือสีเข้ม #1E3A47
configure(validate='none')	ปิดการแก้ค่าอัตโนมัติของ DateEntry: ถ้าพิมพ์ผิด ช่องจะไม่ถูกเปลี่ยนเป็นวันนี้เงียบๆ แต่ check_form() จะเตือนแทน
เริ่มต้นเป็นช่องว่าง	DateEntry ใส่วันนี้ให้เองตอนสร้าง โปรแกรมจึงลบออก ผู้ใช้ต้องเลือกหรือพิมพ์เอง

2.3 แก้ไขข้อมูลผู้ป่วย

ตัวอย่าง: คลิกแถว **วิภาวรรณ ศรีทอง (รหัส 4)** แก้เบอร์จาก 0872223344 เป็น **0879998888** แล้วกด "บันทึกการแก้ไข"

1. on_select() อ่านรหัสจาก iid ของแถว แล้วดึงข้อมูลจากฐานข้อมูลใหม่ขึ้นฟอร์ม (ไม่อ่านค่าจาก Treeview เพราะจะแปลงเบอร์เป็นตัวเลข เลข 0 ข้างหน้าหาย) ข้อความบนฟอร์มเปลี่ยนเป็น "โหมด : แก้ไขข้อมูลผู้ป่วยรหัส 4" และโหลดประวัติการรักษาของคนนี้ (2.4)
2. update_patient() » check_form() ตรวจแบบเดียวกับตอนลงทะเบียน
3. เช็กเบอร์ซ้ำ **โดยไม่นับตัวเอง** (patient_id <> ?)
4. database.execute() รัน UPDATE ได้จำนวนแถวที่ถูกแก้ ถ้าได้ 0 แปลว่าไม่พบผู้ป่วยแล้ว » เตือน ถ้าได้ 1 » "บันทึกสำเร็จ"

SQL ที่รัน

```
SELECT patient_name, phone, birth_date FROM patients WHERE patient_id = ?;   -- on_select()
SELECT patient_name FROM patients WHERE phone = ? AND patient_id <> ?;       -- เบอร์ซ้ำกับคนอื่นไหม
UPDATE patients SET patient_name = ?, phone = ?, birth_date = ?
WHERE patient_id = ?;
```

ตาราง	ก่อน	หลัง
patients (รหัส 4)	4 | วิภาวรรณ ศรีทอง | 0872223344 | 1995-07-09	**4 | วิภาวรรณ ศรีทอง | 0879998888 | 1995-07-09**
appointments (คิวที่ 7)	7 | patient_id 4 | schedule_id 10 | ตรวจแล้ว	7 | patient_id 4 | schedule_id 10 | ตรวจแล้ว (ไม่ต้องแก้)

> **ทำไมคิวไม่ต้องแก้ตาม**
>
> appointments เก็บแค่ patient_id ไม่ได้เก็บเบอร์ซ้ำ แก้เบอร์ที่ patients ที่เดียว ทุกคิวของผู้ป่วยคนนี้ก็ได้เบอร์ใหม่ทันที นี่คือผลของการออกแบบแบบ 3NF

2.4 ประวัติการรักษา

คลิกชื่อผู้ป่วยในตารางด้านบน on_select() จะเรียก load_history(patient_id) แสดงผลการรักษาทุกครั้งของคนนั้นในกล่อง "ประวัติการรักษา" ด้านล่าง เรียงจากล่าสุด ตัวอย่างจากการรันจริง: คลิก **อำนาจ จันทร์เพ็ญ (รหัส 12)** ได้ "อำนาจ จันทร์เพ็ญ มีประวัติการรักษา 3 ครั้ง (เรียงจากล่าสุด)"



ประวัติการรักษาของ อำนาจ จันทร์เพ็ญ (ภาพจากโปรแกรมจริง 24 ตุลาคม 2569)

load_history()

```
SELECT s.work_date, d.doctor_name, dp.dept_name,
       t.diagnosis, IFNULL(t.treatment, '-')          -- การรักษาที่เว้นว่างแสดงเป็น -
FROM treatments t
JOIN appointments a ON t.appt_id      = a.appt_id      -- ผลการรักษา » คิว
JOIN schedules s    ON a.schedule_id = s.schedule_id   -- คิว » วันที่
JOIN doctors d      ON s.doctor_id   = d.doctor_id
JOIN departments dp ON d.dept_id     = dp.dept_id
WHERE a.patient_id = ?                                 -- ผู้ป่วยมาจากคิว ไม่ได้เก็บใน treatments
ORDER BY s.work_date DESC, s.slot_time DESC;
```

วันที่	แพทย์	ผลวินิจฉัย	การรักษา
2026-08-25	นพ.ประวิทย์ ศรีสุข	เบาหวานชนิดที่ 2 ควบคุมได้ดี	ให้ยาเดิมต่อ นัดเจาะเลือดซ้ำใน 3 เดือน
2026-07-26	นพ.เกียรติศักดิ์ บุญมา	ต้อกระจกตาขวา	ผ่าตัดสลายต้อกระจกตาขวาและใส่เลนส์แก้วตาเทียม
2026-04-27	นพ.ประวิทย์ ศรีสุข	เบาหวานชนิดที่ 2	ยา Metformin 500 mg วันละ 2 ครั้ง และควบคุมอาหาร

treatments ไม่ได้เก็บ patient_id เพราะรู้ได้จาก appt_id » appointments.patient_id อยู่แล้ว (3NF) จึงกรองผู้ป่วยที่ a.patient_id ผู้ป่วยที่ยังไม่เคยตรวจ เช่น ชไมพร ที่เพิ่งลงทะเบียน จะขึ้นว่า "ยังไม่มีประวัติการรักษา" ส่วนผู้ป่วยที่มีประวัติหลายรายการในข้อมูลตั้งต้น คือ อำนาจ จันทร์เพ็ญ และ ด.ช.ภูมิ สุขสันต์ คนละ 3 รายการ

ขั้นที่ 3 จองคิว

ส่วนของ C1 · ไฟล์ booking_tab.py

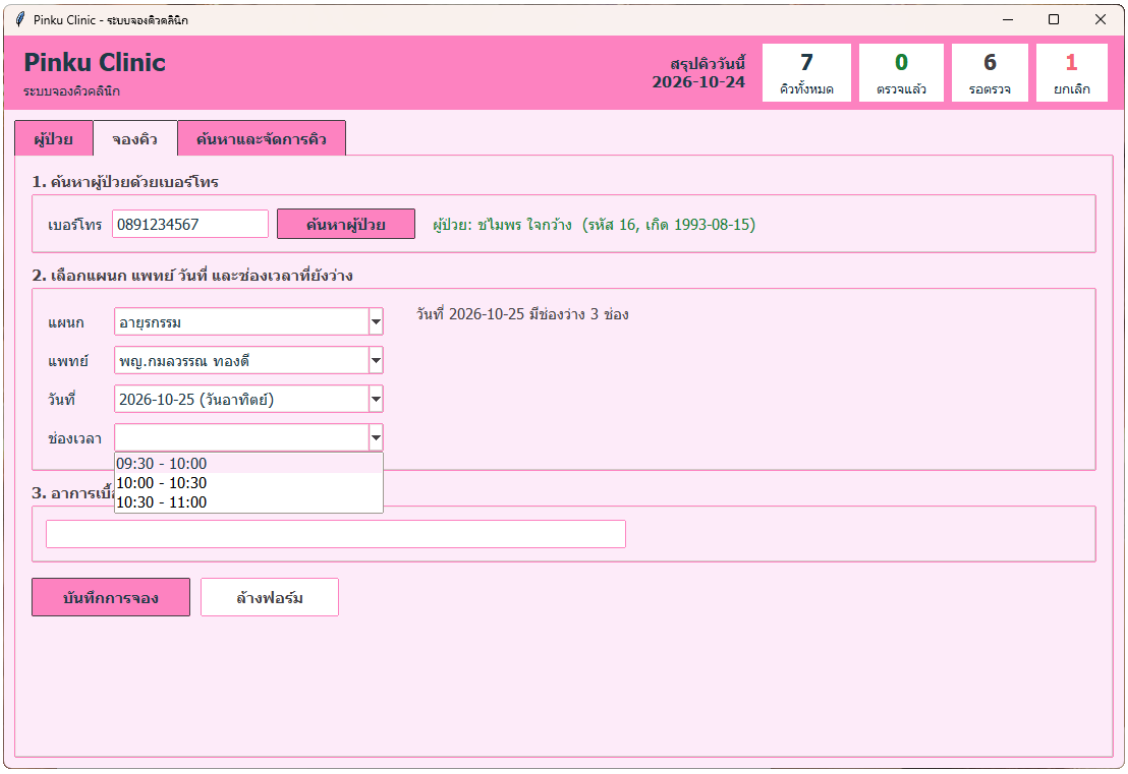
หัวข้อ	สรุป
ผู้ใช้ทำอะไร	ค้นผู้ป่วยด้วยเบอร์โทร » เลือกแผนก » แพทย์ » วันที่ » ช่องเวลาที่ยังว่าง » (กรอกอาการ ไม่บังคับ) » กด "บันทึกการจอง" » ยืนยัน
ฟังก์ชันที่ทำงาน	find_patient() » load_doctors() » load_dates() » load_slots() » save_booking() » database.insert() » header_bar.refresh()
SQL ที่รัน	SELECT หาผู้ป่วย แผนก แพทย์ วันและช่องที่ว่าง » ตรวจซ้ำ 2 คำสั่ง » INSERT INTO appointments
ข้อมูลเปลี่ยน	appointments เพิ่ม 1 แถว status = จองแล้ว · schedules ไม่เปลี่ยน แต่ช่องที่จองไม่แสดงเป็นช่องว่างอีก
ไปต่อ	ขั้นที่ 4 ไปแท็บ "ค้นหาและจัดการคิว" เพื่อดูคิวที่เพิ่งจอง

3.1 เลือกทีละขั้น

ตัวอย่าง: จองให้ **ชไมพร ใจกว้าง** กับ **พญ.กมลวรรณ ทองดี (อายุรกรรม) วันพรุ่งนี้ 2026-10-25 เวลา 10:00**

ผู้ใช้ทำ	ฟังก์ชัน	SQL	ผลที่เห็น (รันจริง)
พิมพ์ 0891234567 กด "ค้นหาผู้ป่วย"	find_patient()	SELECT patient_id, patient_name, birth_date FROM patients WHERE phone = ?	ผู้ป่วย: ชไมพร ใจกว้าง (รหัส 16, เกิด 1993-08-15) ตัวสีเขียว
(แผนกโหลดไว้ตอนสร้างแท็บ)	load_departments()	SELECT dept_id, dept_name FROM departments ORDER BY dept_id	5 แผนก
เลือก "อายุรกรรม"	on_dept_selected() » load_doctors(1)	SELECT doctor_id, doctor_name FROM doctors WHERE dept_id = ? ORDER BY doctor_id	นพ.ประวิทย์ ศรีสุข, พญ.กมลวรรณ ทองดี
เลือก "พญ.กมลวรรณ ทองดี"	on_doctor_selected() » load_dates(2)	SELECT DISTINCT work_date ... (3.2)	2026-10-25 (วันอาทิตย์)
เลือกวันพรุ่งนี้	on_date_selected() » load_slots(2, 2026-10-25)	SELECT schedule_id, slot_time ... (3.2)	09:30 - 10:00, 10:00 - 10:30, 10:30 - 11:00
เลือก 10:00 - 10:30 กด "บันทึกการจอง" » ตกลง	save_booking()	ตรวจซ้ำ แล้ว INSERT (3.3)	หมายเลขคิว: 31

Combobox แสดงแค่ชื่อหรือเวลา แต่โปรแกรมเก็บรายการ (รหัส, ชื่อ) ไว้ใน ui['doctors'] และ ui['slots'] แล้วใช้ current() (ลำดับที่เลือก) หารหัสกลับมา เช่น เลือก 10:00 - 10:30 ซึ่งเป็นตัวเลือกลำดับที่ 1 (current() นับจาก 0) » schedule_id = 24



ช่องเวลาที่ว่างของ พญ.กมลวรรณ ทองดี วันพรุ่งนี้ (ภาพจากโปรแกรมจริง)

3.2 หาช่องเวลาที่ยังว่าง

load_slots() — load_dates() ใช้เงื่อนไขเดียวกัน แต่ SELECT DISTINCT s.work_date

```
SELECT s.schedule_id, s.slot_time
FROM schedules s
LEFT JOIN appointments a ON a.schedule_id = s.schedule_id
                        AND a.status <> 'ยกเลิก'          -- คิวที่ยกเลิกไม่นับ
WHERE s.doctor_id = ? AND s.work_date = ?
  AND (s.work_date > date('now', 'localtime')              -- พรุ่งนี้เป็นต้นไป: ทุกช่อง
       OR (s.work_date = date('now', 'localtime')          -- วันนี้: เฉพาะช่องที่
           AND s.slot_time > strftime('%H:%M', 'now', 'localtime')))   -- ยังไม่ถึงเวลา
  AND a.appt_id IS NULL                                   -- จับคู่คิวไม่ได้ = ว่าง
ORDER BY s.slot_time;
```

LEFT JOIN จับคู่ทุกช่องเวลากับคิวที่ยังไม่ยกเลิก ช่องไหนจับคู่ไม่ได้ (a.appt_id IS NULL) แปลว่าว่าง คิวที่ยกเลิกถูกตัดทิ้งตั้งแต่ในเงื่อนไข ON ช่องนั้นจึงกลับมาว่าง ถ้าเป็นวันนี้ แสดงเฉพาะช่องที่ยังไม่ถึงเวลาปัจจุบัน

schedule_id	slot_time	คิวในช่องนี้	แสดงใน Combobox
22	09:00	คิวที่ 14 สมหญิง ใจดี (จองแล้ว)	ไม่แสดง
23	09:30	คิวที่ 17 พงศกร แสงทอง (ยกเลิก)	**แสดง (คิวยกเลิกไม่นับ)**
24	10:00	—	แสดง
25	10:30	—	แสดง

ตัวอย่างนี้คือช่องของ พญ.กมลวรรณ ทองดี วันพรุ่งนี้ ก่อนจอง

3.3 กด "บันทึกการจอง": ตรวจซ้ำก่อนบันทึก

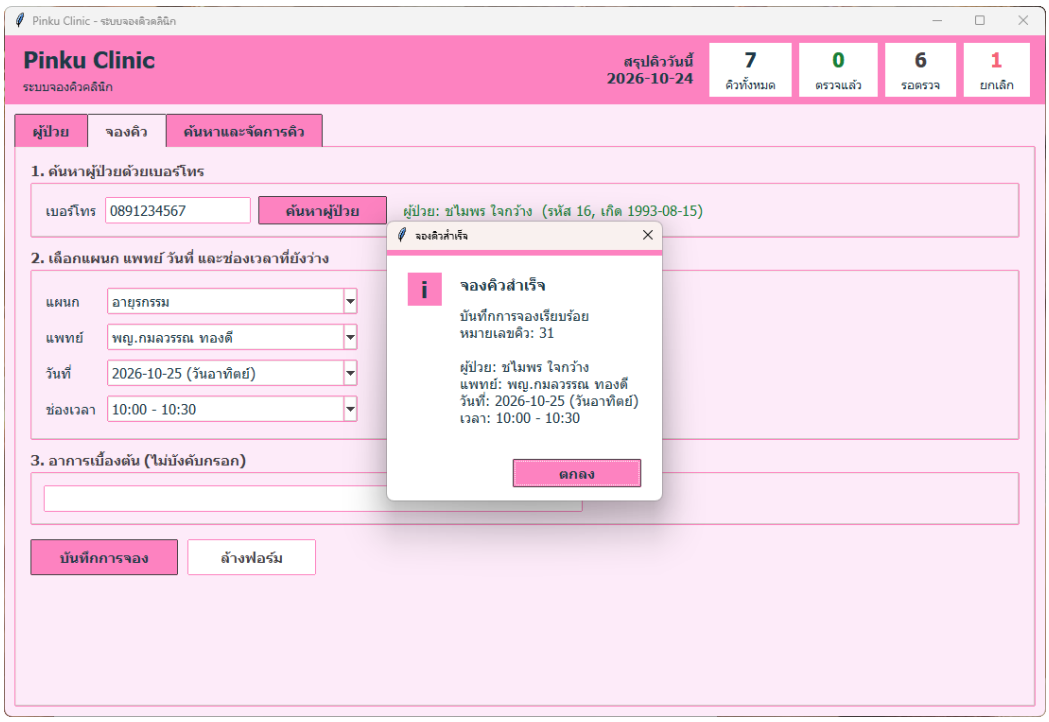
1. หาผู้ป่วยจากเบอร์ในช่องอีกครั้ง (กันกรณีแก้เบอร์หลังกดค้นหา) ไม่พบ » เตือน
2. ต้องเลือกแผนก แพทย์ วันที่ ช่องเวลาครบ ขาดช่องไหน » เตือนทีละช่อง เช่น "กรุณาเลือกช่องเวลา"
3. ช่องที่เลือกต้องยังไม่เลยเวลา: ถ้าเป็นวันนี้ และ slot_time <= เวลาปัจจุบัน » เตือน "ช่องเวลานี้ผ่านไปแล้ว" แล้วโหลดรายการใหม่
4. ช่องต้องยังว่าง: นับคิวที่ยังไม่ยกเลิกในช่องนั้น ถ้ามากกว่า 0 » เตือน "ช่องเวลาไม่ว่าง"
5. ผู้ป่วยต้องไม่มีคิวซ้อน: หาคิวที่ยังไม่ยกเลิกของผู้ป่วยคนนี้ในวันและเวลาเดียวกัน เจอ » เตือน "คิวซ้อนกัน"
6. dialogs.ask_yes_no() ถามยืนยันพร้อมสรุป ผู้ป่วย แพทย์ วันที่ เวลา (และอาการถ้ากรอก)
7. INSERT คิวใหม่ อาการว่างส่ง None (เก็บเป็น NULL) ส่วน status และ booked_at ใช้ค่าเริ่มต้นของตาราง
8. สำเร็จ » แจ้ง "หมายเลขคิว: 31" » load_dates() โหลดใหม่ ช่อง 10:00 หายไปจากรายการ » header_bar.refresh()

SQL ที่รันตอนกดบันทึก

```
SELECT COUNT(*) FROM appointments
WHERE schedule_id = ? AND status <> 'ยกเลิก';              -- ข้อ 4 ช่องยังว่างไหม

SELECT d.doctor_name
FROM appointments a
JOIN schedules s ON a.schedule_id = s.schedule_id
JOIN doctors d   ON s.doctor_id   = d.doctor_id
WHERE a.patient_id = ? AND a.status <> 'ยกเลิก'
  AND s.work_date = ? AND s.slot_time = ?;                 -- ข้อ 5 คิวซ้อนไหม

INSERT INTO appointments (patient_id, schedule_id, symptom)
VALUES (?, ?, ?);                                          -- ข้อ 7 (16, 24, None)
```



กล่อง "จองคิวสำเร็จ" หมายเลขคิว: 31 (ภาพจากโปรแกรมจริง)

ตาราง appointments ก่อนและหลัง

appt_id	patient_id	schedule_id	symptom	status	booked_at
29	13	49	ตรวจภายในประจำปี	ตรวจแล้ว	(ข้อมูลตั้งต้น)
30	7	50	ตรวจติดตามความดัน	ตรวจแล้ว	(ข้อมูลตั้งต้น)
31 (ใหม่)	**16**	**24**	**NULL**	**จองแล้ว**	**วันเวลาที่กดบันทึก**

ถ้าจองซ้อนเวลา

ลองจอง ชไมพร อีกครั้งกับ พญ.ปิยะนุช สายสุวรรณ (สูติ-นรีเวชกรรม) พรุ่งนี้ 10:00 » ข้อ 5 เจอคิวที่ 31 » เตือน: ผู้ป่วย "ชไมพร ใจกว้าง" มีคิวกับ พญ.กมลวรรณ ทองดี วันที่ 2026-10-25 เวลา 10:00 อยู่แล้ว

ถ้าคนสองคนจองช่องเดียวกันพร้อมกัน partial unique index จะปฏิเสธคนที่สอง และ `error_to_thai()` แสดง "ช่องเวลานี้มีผู้ป่วยจองไปแล้ว กรุณาเลือกช่องเวลาอื่น" แล้วโหลดช่องว่างใหม่

ขั้นที่ 4 ค้นหาคิว

ส่วนของ D1 · ไฟล์ search_tab.py

หัวข้อ	สรุป
ผู้ใช้ทำอะไร	เปิดแท็บ (ช่องวันที่ตั้งเป็นวันนี้ไว้แล้ว) กรอกชื่อบางส่วน เบอร์โทร หรือวันที่ YYYY-MM-DD (หรือกด "วันนี้") แล้วกด "ค้นหา" หรือ Enter
ฟังก์ชันที่ทำงาน	search() » read_conditions() » is_valid_date() » load_results() » database.fetch_all()
SQL ที่รัน	SELECT ที่ JOIN appointments กับ patients schedules doctors departments คำสั่งเดียว รองรับทุกแบบการค้น
ข้อมูลเปลี่ยนไหม	ไม่เปลี่ยน อ่านอย่างเดียว
ไปต่อ	คลิกเลือกคิวในตาราง » ขั้นที่ 5

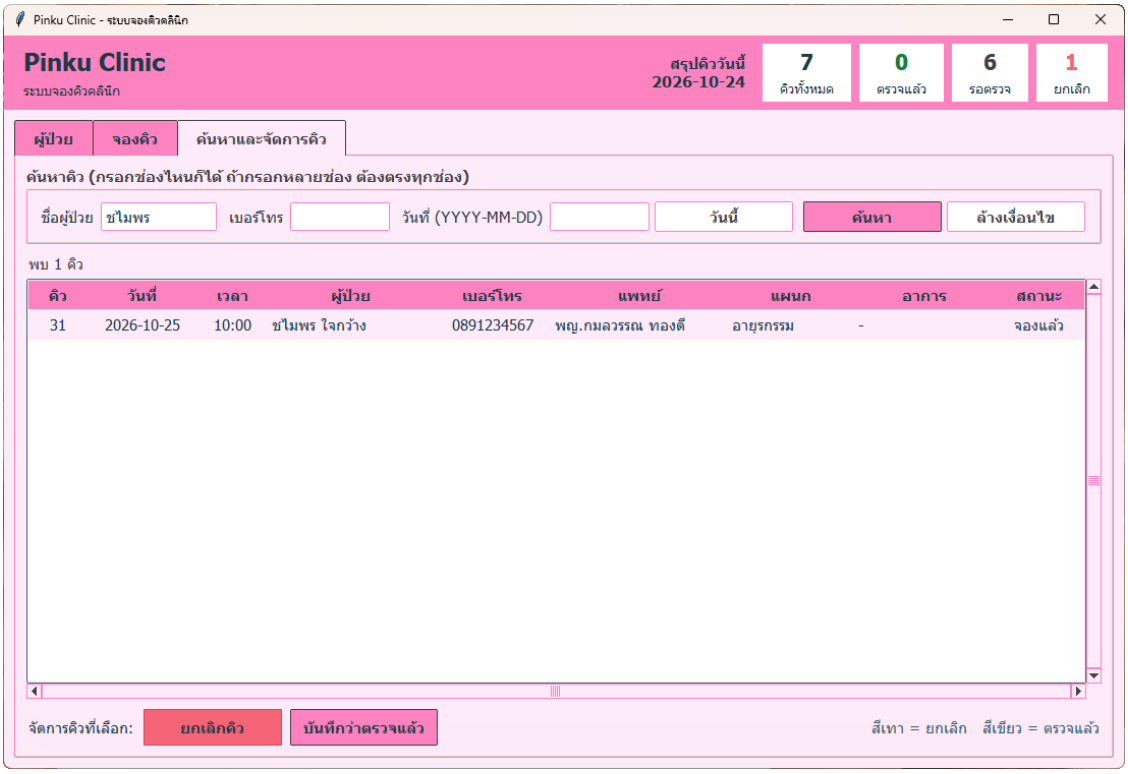
load_results()

```
SELECT a.appt_id, s.work_date, s.slot_time, p.patient_name, p.phone,
       d.doctor_name, dp.dept_name, IFNULL(a.symptom, '-'), a.status
FROM appointments a
JOIN patients p      ON a.patient_id  = p.patient_id
JOIN schedules s     ON a.schedule_id = s.schedule_id
JOIN doctors d       ON s.doctor_id   = d.doctor_id
JOIN departments dp ON d.dept_id      = dp.dept_id
WHERE (? = '' OR p.patient_name LIKE ?)        -- ช่องชื่อว่าง = ไม่กรองชื่อ
  AND (? = '' OR p.phone = ?)                  -- ช่องเบอร์ว่าง = ไม่กรองเบอร์
  AND (? = '' OR s.work_date = ?)              -- ช่องวันที่ว่าง = ไม่กรองวันที่
ORDER BY s.work_date, s.slot_time, d.doctor_name;
```

? ตัวที่	ค่าที่ส่ง	ความหมาย
1	name	ถ้าช่องชื่อว่าง ส่วน ? = '' เป็นจริง เงื่อนไขชื่อจึงไม่มีผล
2	'%' + name + '%'	ชื่อที่มีคำนี้อยู่ตรงไหนก็ได้ (พิมพ์บางส่วนได้)
3, 4	phone, phone	เบอร์ตรงทั้งเบอร์ (ตัด - และช่องว่างแล้ว)
5, 6	work_date, work_date	วันที่ตรงกัน

ตัวอย่างผลการค้นหา (รันจริง)

เงื่อนไข	ผลที่ได้
ชื่อ "ชไมพร" (ช่องอื่นว่าง)	พบ 1 คิว: 31 | 2026-10-25 | 10:00 | ชไมพร ใจกว้าง | 0891234567 | พญ.กมลวรรณ ทองดี | อายุรกรรม | - | จองแล้ว
ชื่อ "สม"	พบ 4 คิว (สมหญิง 2 คิว, สมศักดิ์ 2 คิว)
กด "วันนี้"	พบ 7 คิวของวันนี้ (ข้อมูลชุดตั้งต้น)
วันที่ "22/10/2026"	เตือน "วันที่ไม่ถูกต้อง" ให้กรอกแบบ YYYY-MM-DD หรือกดปุ่ม "วันนี้"
สีของแถวในตาราง	ปกติ = จองแล้ว, **เขียว = ตรวจแล้ว**, **เทา = ยกเลิก** (แต่ละแถวใช้ appt_id เป็น iid)



ค้นหาด้วยชื่อ "ชไมพร" พบคิวที่ 31 (ภาพจากโปรแกรมจริง)

ขั้นที่ 5 ยกเลิกคิว หรือบันทึกว่าตรวจแล้วพร้อมผลวินิจฉัย

ส่วนของ D1 · ไฟล์ search_tab.py และ database.py

หัวข้อ	สรุป
ผู้ใช้ทำอะไร	คลิกเลือกคิวในตาราง แล้วกด "ยกเลิกคิว" หรือ "บันทึกว่าตรวจแล้ว" » ยืนยัน (บันทึกว่าตรวจแล้วต้องกรอกฟอร์มผลวินิจฉัยต่อ)
ฟังก์ชันที่ทำงาน	cancel_appointment() / mark_done() » get_selected_appointment() » ask_treatment() » database.execute() / database.execute_transaction() » load_results() » select_row() » header_bar.refresh()
SQL ที่รัน	SELECT สถานะล่าสุดของคิว » UPDATE appointments (ยกเลิก) หรือ UPDATE appointments + INSERT treatments ใน transaction เดียว (ตรวจแล้ว)
ข้อมูลเปลี่ยน	ยกเลิก: appointments แก้ status 1 แถว ไม่ลบแถว · ตรวจแล้ว: appointments แก้ 1 แถว และ treatments เพิ่ม 1 แถว พร้อมกัน
ไปต่อ	กลับไปขั้นที่ 2 หรือ 3 สำหรับผู้ป่วยรายถัดไป ช่องที่ยกเลิกกลับมาให้จองในแท็บจองคิวทันที

5.1 ยกเลิกคิว

ตัวอย่าง: เลือก **คิวที่ 31 (ชไมพร พรุ่งนี้ 10:00)** แล้วกด "ยกเลิกคิว"

1. get_selected_appointment() อ่าน appt_id จาก iid ของแถว แล้วดึงสถานะล่าสุดจากฐานข้อมูล ไม่ได้เลือกแถว » เตือน "กรุณาคลิกเลือกคิวในตารางก่อน"
2. สถานะต้องเป็น "จองแล้ว" ไม่ใช่ » เตือน "ยกเลิกไม่ได้" พร้อมบอกสถานะปัจจุบัน
3. ask_yes_no(..., icon='warning') ถามยืนยันด้วยสีเน้น พร้อมรายละเอียดคิว และบอกว่าหลังยกเลิก ช่องเวลานี้จะว่างให้คนอื่นจองได้
4. UPDATE โดยมีเงื่อนไข status = 'จองแล้ว' ซ้อนอีกชั้น ได้ 0 แถว » เตือน "สถานะของคิวนี้ถูกเปลี่ยนไปแล้ว" ได้ 1 แถว » "ยกเลิกคิวที่ 31 เรียบร้อย"
5. load_results() โหลดตารางใหม่ แล้ว select_row() ไฮไลต์คิวเดิม ให้เห็นแถวเปลี่ยนเป็นสีเทา กดยกเลิกซ้ำ » "ยกเลิกไม่ได้ ... มีสถานะ "ยกเลิก" แล้ว"

SQL ที่รัน

```
SELECT a.appt_id, a.status, s.work_date, s.slot_time, p.patient_name, d.doctor_name
FROM appointments a
JOIN patients p  ON a.patient_id  = p.patient_id
JOIN schedules s ON a.schedule_id = s.schedule_id
JOIN doctors d   ON s.doctor_id   = d.doctor_id
WHERE a.appt_id = ?;                                  -- get_selected_appointment()

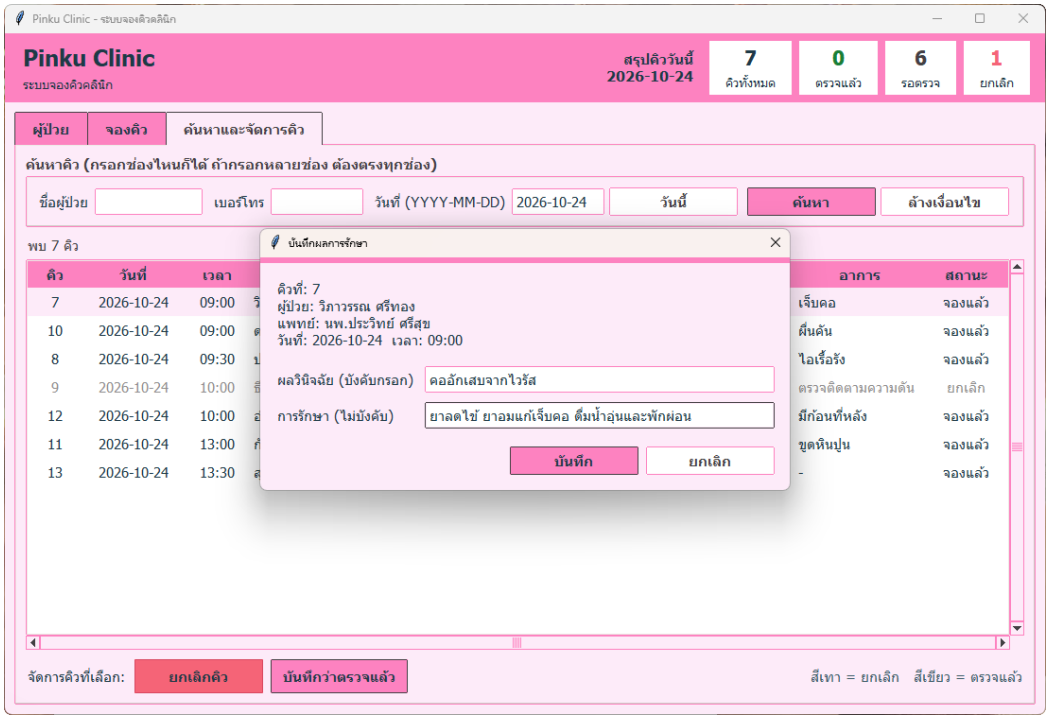
UPDATE appointments SET status = 'ยกเลิก'             -- ไม่ลบแถว เก็บเป็นประวัติ
WHERE appt_id = ? AND status = 'จองแล้ว';
```

ตาราง	ก่อน	หลัง
appointments (คิวที่ 31)	31 | 16 | 24 | NULL | จองแล้ว	**31 | 16 | 24 | NULL | ยกเลิก (แถวยังอยู่)**
ช่อง 10:00 พรุ่งนี้ ในแท็บจองคิว	ไม่แสดง (มีคิว 31)	**แสดงอีกครั้ง: 09:30 - 10:00, 10:00 - 10:30, 10:30 - 11:00**

5.2 บันทึกว่าตรวจแล้ว: ยืนยัน » ฟอร์มผลวินิจฉัย » transaction

ตัวอย่าง: กด "วันนี้" เลือก **คิวที่ 7 (วิภาวรรณ ศรีทอง กับ นพ.ประวิทย์ ศรีสุข วันนี้ 09:00 อาการ "เจ็บคอ")** แล้วกด "บันทึกว่าตรวจแล้ว"

1. mark_done() » get_selected_appointment() อ่านสถานะล่าสุด สถานะต้องเป็น "จองแล้ว" ไม่ใช่ » เตือน "บันทึกไม่ได้"
2. วันนัดต้องเป็นวันนี้หรือผ่านมาแล้ว ถ้าเป็นวันข้างหน้า » เตือน "ยังไม่ถึงวันนัด"
3. ask_yes_no() ถาม "บันทึกว่าคิวนี้ตรวจแล้วใช่ไหม?" กดตกลง » ask_treatment() เปิดฟอร์ม "บันทึกผลการรักษา"
4. ฟอร์มมี 2 ช่อง: **ผลวินิจฉัย (บังคับกรอก)** และ **การรักษา (ไม่บังคับ)** กดบันทึกตอนผลวินิจฉัยว่าง » เตือน "กรุณากรอกผลวินิจฉัยก่อนบันทึก" ช่องการรักษาที่ว่างส่งเป็น None (NULL) กดยกเลิกหรือ Esc » ไม่บันทึกอะไรเลย
5. database.execute_transaction() รัน 2 คำสั่งในการเชื่อมต่อเดียว: UPDATE สถานะ และ INSERT ผลการรักษา สำเร็จทั้งคู่ถึงจะ commit
6. แจ้ง "บันทึกว่าคิวที่ 7 ตรวจแล้ว เรียบร้อย / ผลวินิจฉัย: คออักเสบจากไวรัส" » แถวเปลี่ยนเป็นสีเขียว » แถบหัวเปลี่ยนเป็น คิวทั้งหมด 7 · ตรวจแล้ว 1 · รอตรวจ 5 · ยกเลิก 1



ฟอร์มผลวินิจฉัยของคิวที่ 7 (ภาพจากโปรแกรมจริง)

mark_done() และ execute_transaction()

```
commands = [
    ("""UPDATE appointments SET status = 'ตรวจแล้ว'
        WHERE appt_id = ? AND status = 'จองแล้ว' """, (7,)),
    ("""INSERT INTO treatments (appt_id, diagnosis, treatment)
        VALUES (?, ?, ?)""", (7, 'คออักเสบจากไวรัส', 'ยาลดไข้ ยาอมแก้เจ็บคอ ดื่มน้ำอุ่นและพักผ่อน')),
]
saved = database.execute_transaction(commands)

def execute_transaction(commands):            # database.py
    conn = get_connection()
    cursor = conn.cursor()
    try:
        for sql, params in commands:
            cursor.execute(sql, params)
            if cursor.rowcount == 0:          # UPDATE ไม่เจอคิวที่ยังจองแล้ว
                conn.rollback()               # ยกเลิกทุกคำสั่งในชุดนี้
                return False
        conn.commit()                         # สำเร็จทั้งชุด ถึงจะบันทึกจริง
        return True
    except sqlite3.Error:
        conn.rollback()                       # เช่น UNIQUE บน treatments.appt_id
        raise
    finally:
        conn.close()
```

ตาราง appointments และ treatments ก่อนและหลัง

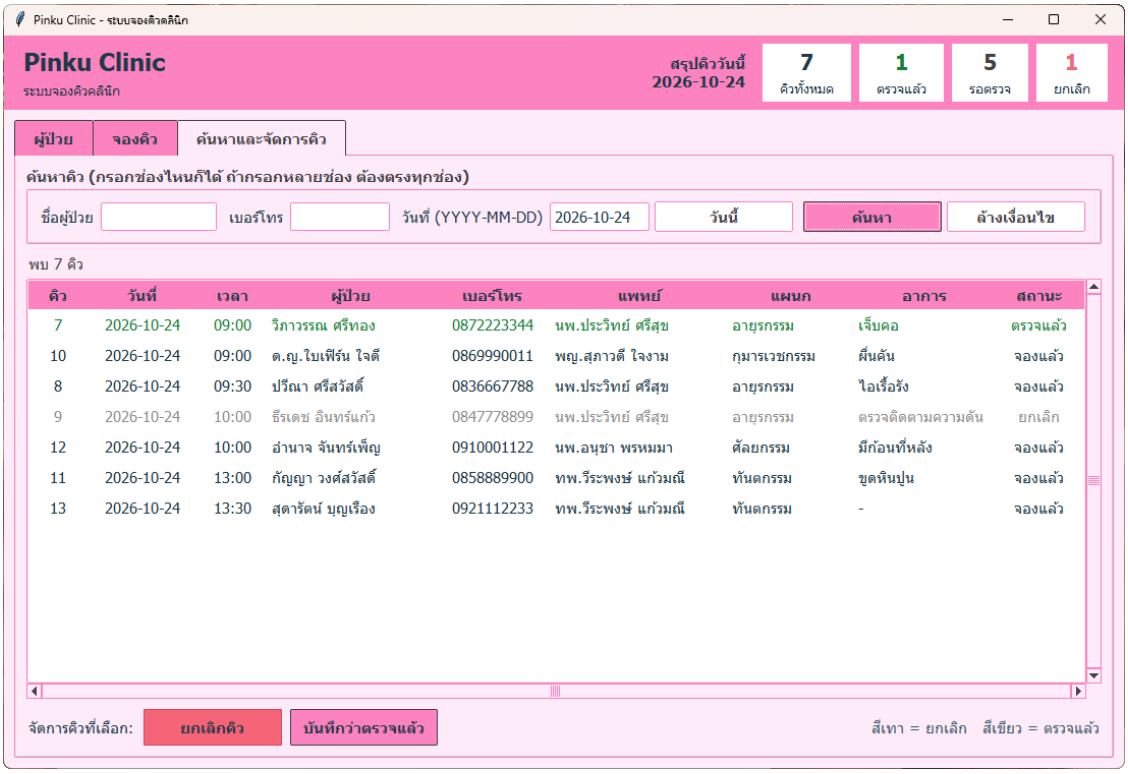
ตาราง	ก่อน	หลัง
appointments (คิวที่ 7)	7 | 4 | 10 | เจ็บคอ | จองแล้ว	**7 | 4 | 10 | เจ็บคอ | ตรวจแล้ว**
treatments	15 แถว (treatment_id 1–15)	**16 | appt_id 7 | คออักเสบจากไวรัส | ยาลดไข้ ยาอมแก้เจ็บคอ ดื่มน้ำอุ่นและพักผ่อน | วันเวลาที่บันทึก**
แถบหัว (วันนี้)	7 / 0 / 6 / 1	**7 / 1 / 5 / 1**

ทำไมต้องเป็น transaction

กฎของระบบ: **ทุกคิวที่ตรวจแล้วต้องมีผลการรักษา** ถ้าทำแยก 2 ครั้ง แล้ว INSERT ล้มหลัง UPDATE สำเร็จ จะได้คิว "ตรวจแล้ว" ที่ไม่มีผลการรักษา

ใน transaction ถ้าคำสั่งใดผิด ทั้งชุดจะถูก rollback ทดสอบจริงบนเครื่อง: ส่ง diagnosis เป็น NULL ฐานข้อมูลปฏิเสธด้วย NOT NULL แล้วคิว 7 ยังเป็น "จองแล้ว" และ treatments ไม่มีแถวของคิว 7 เลย

ถ้ามีคนเปลี่ยนสถานะคิวไปก่อน UPDATE จะแก้ 0 แถว execute_transaction() rollback แล้วคืน False โปรแกรมแจ้ง "สถานะของคิวนี้ถูกเปลี่ยนไปแล้ว" จึงไม่มีผลการรักษาหลุดเข้าไปโดยไม่มีคิว



หลังบันทึก: คิวที่ 7 เป็นสีเขียว และแถบหัวเปลี่ยนเป็น คิวทั้งหมด 7 · ตรวจแล้ว 1 · รอตรวจ 5 · ยกเลิก 1 (ภาพจากโปรแกรมจริง)

ปุ่มไหนกดได้ ตามสถานะของคิว

สถานะปัจจุบัน	ยกเลิกคิว	บันทึกว่าตรวจแล้ว
จองแล้ว	ได้	ได้ ถ้าวันนัดเป็นวันนี้หรือผ่านมาแล้ว (และต้องกรอกผลวินิจฉัย)
ตรวจแล้ว	ไม่ได้	ไม่ได้
ยกเลิก	ไม่ได้	ไม่ได้

8. เมื่อฐานข้อมูลปฏิเสธคำสั่ง: error_to_thai()

ส่วนของ B1 · ไฟล์ database.py

โปรแกรมตรวจข้อมูลก่อนบันทึกเสมอ ข้อบังคับในฐานข้อมูลจึงเป็นด่านสุดท้าย ถ้าฐานข้อมูลปฏิเสธ ทุกแท็บจับ sqlite3.Error แล้วส่งให้ error_to_thai() แปลงเป็นข้อความที่คนทั่วไปเข้าใจ แล้วแสดงด้วย dialogs.show_error() **ไม่มีข้อความ error ภาษาอังกฤษขึ้นบนหน้าจอโปรแกรม** ข้อความภาษาอังกฤษตัวจริงพิมพ์ไว้ใน Terminal เท่านั้น สำหรับคนเขียนโปรแกรม

รูปแบบที่ทุกแท็บใช้ (ตัวอย่างจาก booking_tab.py)

```
try:
    new_id = database.insert(sql, (patient_id, schedule_id, symptom))
except sqlite3.Error as error:
    dialogs.show_error('จองไม่สำเร็จ', database.error_to_thai(error))
    load_slots(doctor_id, work_date)
    return
```

error จาก SQLite มีคำว่า	ข้อความที่ผู้ใช้เห็น
UNIQUE constraint failed: patients.phone	เบอร์โทรนี้มีผู้ป่วยลงทะเบียนไว้แล้ว กรุณาตรวจสอบเบอร์อีกครั้ง
UNIQUE constraint failed: appointments.schedule_id	ช่องเวลานี้มีผู้ป่วยจองไปแล้ว กรุณาเลือกช่องเวลาอื่น
UNIQUE constraint failed: treatments.appt_id	คิวนี้มีการบันทึกผลการรักษาไว้แล้ว ไม่สามารถบันทึกซ้ำได้
UNIQUE constraint failed (อื่นๆ)	ข้อมูลนี้มีอยู่ในระบบแล้ว ไม่สามารถบันทึกซ้ำได้
FOREIGN KEY constraint failed	ทำรายการไม่ได้ เพราะข้อมูลนี้ยังเชื่อมโยงกับข้อมูลอื่นอยู่ หรือข้อมูลที่อ้างถึงถูกลบไปแล้ว
CHECK constraint failed + phone	เบอร์โทรต้องเป็นตัวเลข 10 หลัก และขึ้นต้นด้วย 0
CHECK constraint failed + birth_date	วันเกิดไม่ถูกต้อง กรุณากรอกแบบ YYYY-MM-DD เป็นปี ค.ศ.
CHECK constraint failed (อื่นๆ)	ข้อมูลบางช่องไม่อยู่ในรูปแบบที่ระบบรับได้
NOT NULL constraint failed	กรุณากรอกข้อมูลให้ครบทุกช่องที่จำเป็น
database is locked	ฐานข้อมูลกำลังถูกใช้งานโดยโปรแกรมอื่น เช่น DB Browser ที่ยังไม่ได้กด Write Changes
no such table	ไม่พบตารางในฐานข้อมูล กรุณารัน python create_db.py ก่อน
อื่นๆ ที่ไม่ได้คาดไว้	เกิดข้อผิดพลาดกับฐานข้อมูล บันทึกไม่สำเร็จ กรุณาลองใหม่อีกครั้ง

9. ตัวอย่าง SQL 10 ข้อใน sql_examples.py

ส่วนของ B1 · ผลจากการรันจริงหลัง create_db.py (23 ตุลาคม 2569)

python sql_examples.py รันทีละตัวอย่าง กด Enter เพื่อไปข้อถัดไป ไฟล์นี้แก้ข้อมูลจริง (ยกเลิกคิว 7 จองช่องใหม่ แก้เบอร์ ลบช่องเวลา) จึงต้องรัน python create_db.py อีกครั้งก่อน Demo

ข้อ	เรื่อง	ผลที่ได้
1	คิวของวันนี้ (JOIN appointments กับ patients schedules doctors departments)	7 แถว
2	จำนวนคิวของแพทย์แต่ละคน (LEFT JOIN + GROUP BY ไม่นับคิวที่ยกเลิก)	ตารางด้านล่าง 10 แถว รวม 27 คิว
3	ช่องเวลาที่ยังว่าง แผนกอายุรกรรม	7 แถว
4	ประวัติคิวของผู้ป่วย ค้นด้วยเบอร์โทร	2 แถว
5	ค้นหาคิวด้วยชื่อบางส่วน	4 แถว
6	UPDATE ยกเลิกคิว 7	แก้ไป 1 แถว · ยกเลิกซ้ำ แก้ไป 0 แถว
7	จองช่องเดิมใหม่หลังยกเลิก	**จองสำเร็จ ได้คิวใหม่ appt_id = 31 · ช่องที่ 10 มี 2 แถว (7 ยกเลิก, 31 จองแล้ว) · จองช่องที่มีคนจองแล้ว » UNIQUE constraint failed**
8	UPDATE แก้เบอร์โทรผู้ป่วย	แก้ไป 1 แถว · แก้เป็นเบอร์ที่มีคนใช้ » UNIQUE constraint failed
9	DELETE ลบช่องเวลาที่ไม่มีใครจอง	ลบไป 2 แถว
10	DELETE ลองลบผู้ป่วยที่มีคิวแล้ว	ลบไม่ได้: FOREIGN KEY constraint failed

ผลตัวอย่างที่ 2: จำนวนคิวของแพทย์แต่ละคน

แพทย์	แผนก	total_appts
นพ.ประวิทย์ ศรีสุข	อายุรกรรม	6
พญ.กมลวรรณ ทองดี	อายุรกรรม	4
ทพ.วีระพงษ์ แก้วมณี	ทันตกรรม	4
พญ.สุภาวดี ใจงาม	กุมารเวชกรรม	3
นพ.ธนากร วงศ์ใหญ่	กุมารเวชกรรม	2
ทพญ.วรรณา แก้วงาม	ทันตกรรม	2
นพ.อนุชา พรหมมา	ศัลยกรรม	2
นพ.เกียรติศักดิ์ บุญมา	ศัลยกรรม	2
พญ.ปิยะนุช สายสุวรรณ	สูติ-นรีเวชกรรม	2
พญ.รัตนา ศรีวงศ์	สูติ-นรีเวชกรรม	0

10. คำถามที่อาจารย์น่าจะถาม (32 ข้อ)

> แยกตามส่วนที่แต่ละคนพูด แต่ **ทุกคนต้องอ่านทั้งหมด** ตอบสั้นๆ ก่อน แล้วยกตัวอย่างจากระบบของเรา ถ้าอาจารย์ถามต่อค่อยลงรายละเอียด

A1 · เปิดเรื่อง ขอบเขต เครื่องมือ หน้าตาโปรแกรม และสรุป

ข้อ 1 ระบบนี้แก้ปัญหาอะไร และใครเป็นผู้ใช้?

แก้ปัญหาการจองคิวด้วยสมุดหรือกระดาษ ซึ่งจองซ้อนกันง่าย ไม่รู้ว่าช่องไหนยังว่าง และหาประวัติคิวกับผลการรักษายาก

ผู้ใช้คือเจ้าหน้าที่เคาน์เตอร์ 1 คน ผู้ป่วยโทรมาหรือมาที่เคาน์เตอร์ เจ้าหน้าที่ลงทะเบียน จองคิว ค้นหา ยกเลิก และบันทึกว่าตรวจแล้วพร้อมผลวินิจฉัยให้

ข้อ 2 ทำไมไม่มีหน้า Login?

งานนี้มีผู้ใช้คนเดียวบนเครื่องเดียว และโจทย์เน้นการออกแบบฐานข้อมูลกับ SQL จึงตัด Login ออก เพื่อให้เวลากับส่วนหลัก

ถ้าใช้จริงควรเพิ่มตาราง users เก็บรหัสผ่านแบบ hash แยกสิทธิ์เจ้าหน้าที่กับแพทย์ และบันทึกว่าใครทำรายการไหน

ข้อ 3 SQLite ต่างจาก MySQL อย่างไร ทำไมเลือก SQLite?

SQLite เป็นไลบรารีที่อ่านเขียนไฟล์เดียว (clinic.db) ไม่ต้องติดตั้งหรือเปิดเซิร์ฟเวอร์ มากับ Python เลย ส่วน MySQL เป็นแบบ client-server ต้องมีโปรแกรมเซิร์ฟเวอร์ บัญชีผู้ใช้ และสิทธิ์ เหมาะกับหลายคนใช้พร้อมกันผ่านเครือข่าย

ข้อต่างที่เจอจริงในงานนี้: SQLite ปิด foreign key เป็นค่าเริ่มต้น, เขียนได้ทีละการเชื่อมต่อ, ไม่มีชนิด DATE แยก (เก็บวันที่เป็น TEXT)

ข้อ 4 ถ้าจะใช้จริงในคลินิก ต้องเพิ่มอะไร?

Login และสิทธิ์ผู้ใช้, หน้าจัดการตารางออกตรวจของแพทย์ (ตอนนี้ช่องเวลาใส่จาก clinic_db.sql), ย้ายไปฐานข้อมูลแบบเซิร์ฟเวอร์ถ้ามีหลายเคาน์เตอร์, สำรองไฟล์ฐานข้อมูลทุกวัน และแจ้งเตือนผู้ป่วยก่อนวันนัด

ข้อ 5 หน้าต่างแจ้งเตือนแบบใหม่ต่างจาก messagebox อย่างไร?

messagebox ของ tkinter ใช้หน้าตาของ Windows เปลี่ยนสีและฟอนต์ไม่ได้ เราจึงเขียน dialogs.py สร้างหน้าต่างเองด้วย Toplevel: แถบสีบอกชนิด (ชมพู = ข้อมูล/ถาม, ชมพูแดง = เตือน/ผิดพลาด) ฟอนต์ Tahoma และวางตรงกลางหน้าต่างหลัก

ทำงานเหมือน messagebox: ล็อกหน้าต่างอื่นด้วย grab_set() รอคำตอบด้วย wait_window() และคืนค่า True/False Enter = ตกลง Esc = ยกเลิก การยืนยันยกเลิกคิวใช้สีเน้นให้ผู้ใช้ระวัง

ข้อ 6 แถบหัวที่มีตัวเลข 4 ช่องมีไว้ทำไม?

ให้เจ้าหน้าที่เห็นภาพรวมของวันนี้ทันทีโดยไม่ต้องค้นหา: คิวทั้งหมด ตรวจแล้ว รอตรวจ และยกเลิก เช่น หลังรีเซ็ตเป็น 7 / 0 / 6 / 1 ตัวเลขอัปเดตทุกครั้งที่จอง ยกเลิก บันทึกว่าตรวจแล้ว และสลับแท็บ

B1 · ฐานข้อมูล กฎของระบบ SQL และการเชื่อมต่อ

ข้อ 7 ทำไมแบ่งเป็น 6 ตาราง และทำไมถึงอยู่ในรูป 3NF?

แยกตามสิ่งที่ระบบต้องจำ ถ้าเก็บรวมตารางเดียว ชื่อ เบอร์ วันเกิดของผู้ป่วยจะซ้ำทุกครั้งที่จอง เปลี่ยนเบอร์ต้องแก้หลายแถว ลงทะเบียนคนที่ยังไม่จองไม่ได้ และลบคิวแล้วข้อมูลผู้ป่วยหายไปด้วย

ทุกตารางมีคีย์หลักฟิลด์เดียว (ผ่าน 2NF) และไม่มีฟิลด์ที่ขึ้นกับฟิลด์ธรรมดาอื่น เช่น doctors เก็บ dept_id ไม่เก็บชื่อแผนก, appointments เก็บแค่ schedule_id, treatments เก็บแค่ appt_id (ผ่าน 3NF)

ข้อ 8 ทำไมแยก treatments เป็นอีกตาราง ไม่เพิ่มคอลัมน์ผลวินิจฉัยใน appointments?

ผลการรักษามีเฉพาะคิวที่ตรวจแล้ว ข้อมูลตั้งต้นมีคิว 30 คิว แต่ผลการรักษา 15 แถว ถ้าใส่ใน appointments อีก 15 คิวที่รอตรวจหรือยกเลิกจะมีช่อง diagnosis, treatment, recorded_at ว่างเป็น NULL และบังคับ NOT NULL ให้ diagnosis ไม่ได้

แยกตารางแล้ว diagnosis เป็น NOT NULL ได้ และความสัมพันธ์ "1 คิว ต่อ 0..1 ผลการรักษา" บังคับด้วย UNIQUE บน treatments.appt_id ทดสอบจริงแล้ว บันทึกผลซ้ำคิวเดิมถูกปฏิเสธด้วย UNIQUE constraint failed: treatments.appt_id

ข้อ 9 ทำไม treatments ไม่เก็บ patient_id ทั้งที่ประวัติการรักษาค้นตามผู้ป่วย?

ผู้ป่วยของผลการรักษารู้ได้จาก appt_id » appointments.patient_id อยู่แล้ว ถ้าเก็บซ้ำจะเกิด transitive dependency (treatment_id » appt_id » patient_id) ผิด 3NF และอาจเกิดกรณีผลการรักษาบอกผู้ป่วย A แต่คิวเป็นของผู้ป่วย B

ประวัติการรักษาจึงใช้ JOIN treatments กับ appointments แล้วกรอง `WHERE a.patient_id = ?` ได้ผลถูกต้องเสมอ

ข้อ 10 ทำไมต้องมีตาราง schedules แยกจาก appointments?

ช่องเวลาต้องมีอยู่ก่อนจะมีคนจอง ระบบถึงจะรู้ว่าช่องไหนว่างเอามาใส่ Combobox ได้ และเมื่อคิวถูกยกเลิก ช่องเวลายังอยู่ให้จองใหม่

ข้อ 11 กันคิวซ้ำอย่างไร?

กัน 2 ชั้น ชั้นโปรแกรม: Combobox แสดงเฉพาะช่องที่ว่าง และตอนกดบันทึกนับคิวในช่องนั้นซ้ำอีกครั้ง ชั้นฐานข้อมูล: UNIQUE INDEX บน appointments(schedule_id) เฉพาะแถวที่ status ไม่ใช่ 'ยกเลิก' (partial index)

ส่วนแพทย์คนเดียวกันห้ามมีช่องเวลาซ้ำ กันด้วย UNIQUE (doctor_id, work_date, slot_time) ในตาราง schedules

ข้อ 12 ทำไมยกเลิกแล้วไม่ลบแถว?

เพื่อเก็บประวัติ และนับสถิติได้ เช่น ตัวเลข "ยกเลิก" บนแถบหัว เราจึงเปลี่ยน status เป็น 'ยกเลิก' แทน

partial index ไม่นับแถวที่ยกเลิก ช่องนั้นจึงจองใหม่ได้ ตัวอย่างที่ 7 ใน sql_examples.py จองช่องที่ 10 ใหม่ได้คิว 31

ข้อ 13 transaction คืออะไร และใช้ตรงไหนในโปรแกรม?

transaction คือการรวมหลายคำสั่งให้เป็นหน่วยเดียว สำเร็จทั้งหมดแล้ว commit หรือถ้าคำสั่งใดผิด ก็ rollback กลับเหมือนไม่เคยทำ

ใช้ตอนบันทึกว่าตรวจแล้ว: UPDATE appointments กับ INSERT treatments อยู่ใน `execute_transaction()` เดียวกัน เพื่อรักษากฎ "ทุกคิวที่ตรวจแล้วต้องมีผลการรักษา" ทดสอบแล้ว ถ้า INSERT ล้ม คิวยังเป็น "จองแล้ว" ไม่มีครึ่งๆ กลางๆ

ข้อ 14 SQL ของตัวเลขสรุปวันนี้ทำงานอย่างไร ทำไมไม่ใช้หลายคำสั่ง?

คำสั่งเดียว: JOIN appointments กับ schedules (วันที่อยู่ที่ schedules) กรอง work_date = วันนี้ แล้วใช้ COUNT(*) นับทั้งหมด และ SUM(CASE WHEN status = ... THEN 1 ELSE 0 END) นับแยกแต่ละสถานะ ได้ 4 ตัวเลขในแถวเดียว (7 = 0 + 6 + 1)

คำสั่งเดียวเปิดการเชื่อมต่อครั้งเดียวและตัวเลขทั้ง 4 มาจากข้อมูลชุดเดียวกัน IFNULL(..., 0) ทำให้วันที่ไม่มีคิวแสดง 0 ไม่ใช่ว่างเปล่า

ข้อ 15 ทำไมต้องสั่ง PRAGMA foreign_keys = ON?

SQLite ปิดการตรวจ foreign key เป็นค่าเริ่มต้น และค่านี้ติดอยู่กับการเชื่อมต่อแต่ละครั้ง ไม่ได้เก็บในไฟล์ ถ้าไม่สั่ง เราจะลบผู้ป่วยที่ยังมีคิว หรือลบคิวที่มีผลการรักษาได้

get_connection() จึงสั่งทุกครั้งที่เชื่อมต่อ ตัวอย่างที่ 10 ใน sql_examples.py แสดงว่าการลบสมหญิงที่มีคิวอยู่ถูกปฏิเสธ

ข้อ 16 ทำไมใช้ ? แทนการต่อสตริง?

ถ้าต่อสตริง เช่น "... WHERE phone = '" + phone + "'" แล้วมีคนพิมพ์ ' OR '1'='1 ลงช่องเบอร์ เงื่อนไขจะเป็นจริงทุกแถว นี่คือ SQL injection

เมื่อใช้ ? ค่าจะถูกส่งแยกจากคำสั่ง ฐานข้อมูลมองเป็นข้อมูลเสมอ ไม่ใช่คำสั่ง

ข้อ 17 ทำไมวันที่และเบอร์โทรเก็บเป็น TEXT?

SQLite ไม่มีชนิด DATE แยก จึงเก็บเป็นข้อความ YYYY-MM-DD ซึ่งใช้กับฟังก์ชัน date() ได้ และเรียงตามตัวอักษรแล้วตรงกับลำดับเวลา

เบอร์โทรเก็บเป็น TEXT เพราะถ้าเป็น INTEGER เลข 0 ข้างหน้าจะหาย ส่วนรูปแบบใช้ CHECK กันไว้ เช่น date(birth_date) IS birth_date ปฏิเสธวันที่ไม่มีจริงอย่าง 2026-02-30

C1 · ลงทะเบียนผู้ป่วย ปฏิทินวันเกิด ประวัติการรักษา และจองคิว

ข้อ 18 ตรวจข้อมูลผู้ป่วยอะไรบ้างก่อนบันทึก?

ช่องว่าง, เบอร์ต้องเป็นเลข 0–9 ครบ 10 หลักขึ้นต้นด้วย 0, วันเกิดต้องเป็นวันที่จริงและไม่อยู่ในอนาคต, เบอร์ห้ามซ้ำกับคนอื่น

ไม่ใช้ isdigit() เพราะรับเลขไทย ๐–๙ ด้วย และถ้ากรอกปี พ.ศ. วันเกิดจะกลายเป็นอนาคต โปรแกรมจะเตือนให้ลบ 543

ข้อ 19 ทำไมใช้ tkcalendar และต้องติดตั้งอะไรเพิ่ม?

Tkinter ไม่มีปฏิทินในตัว tkcalendar ให้ DateEntry ที่เป็นช่องกรอกพร้อมปฏิทินเลื่อนลง ต้องติดตั้ง 1 ครั้งด้วย python -m pip install tkcalendar (ได้ babel มาด้วย) และมีอยู่ใน requirements.txt

ถ้าเครื่องไม่มี tkcalendar โปรแกรมจะไม่ล่ม แต่ขึ้นกล่อง "ยังไม่ได้ติดตั้ง tkcalendar" พร้อมคำสั่งติดตั้ง แล้วปิดตัวเอง

ข้อ 20 ปฏิทินเป็นภาษาไทยแต่ปีเป็น ค.ศ. ได้อย่างไร และพิมพ์วันเกิดเองได้ไหม?

ตั้ง locale='th_TH' ให้ชื่อเดือนและวันเป็นภาษาไทย และ date_pattern='yyyy-mm-dd' ให้ได้ปี ค.ศ. รูปแบบเดียวกับที่ฐานข้อมูลตรวจ ตั้ง maxdate เป็นวันนี้ จึงเลือกวันในอนาคตจากปฏิทินไม่ได้

พิมพ์เองได้ เราปิด validate ของ DateEntry เพราะค่าเดิมจะเปลี่ยนวันที่ที่พิมพ์ผิดเป็นวันนี้โดยไม่บอก ให้ check_form() เตือนแทน

ข้อ 21 ประวัติการรักษาดึงมาอย่างไร?

คลิกชื่อผู้ป่วย » load_history() รัน SELECT จาก treatments JOIN appointments schedules doctors departments กรองด้วย a.patient_id เรียงวันที่ล่าสุดก่อน ตัวอย่าง อำนาจ จันทร์เพ็ญ ได้ 3 รายการ

หลังบันทึกว่าคิวของผู้ป่วยคนไหนตรวจแล้ว ประวัติของคนนั้นจะมีแถวใหม่ทันที เพราะอ่านจากตาราง treatments ทุกครั้งที่คลิก

ข้อ 22 ช่องเวลาว่างหามาอย่างไร?

ใช้ LEFT JOIN schedules กับ appointments โดยจับคู่เฉพาะคิวที่ยังไม่ยกเลิก ช่องไหนจับคู่ไม่ได้ (a.appt_id IS NULL) แปลว่าว่าง ถ้าใช้ INNER JOIN จะเหลือแต่ช่องที่มีคิว จึงหาช่องว่างไม่ได้

ข้อ 23 ถ้าสองคนจองช่องเดียวกันพร้อมกันจะเกิดอะไร?

คนที่บันทึกก่อนได้คิว คนที่สองจะเจอการตรวจตอนกดบันทึก ถ้าหลุดผ่านมาได้จริง partial unique index ในฐานข้อมูลจะปฏิเสธ แล้วโปรแกรมแสดง "ช่องเวลานี้มีผู้ป่วยจองไปแล้ว กรุณาเลือกช่องเวลาอื่น"

ข้อ 24 ผู้ป่วยคนเดียวจองสองแพทย์ในเวลาเดียวกันได้ไหม?

ไม่ได้ ก่อนบันทึกโปรแกรมหาคิวที่ยังไม่ยกเลิกของผู้ป่วยในวันและเวลาเดียวกัน เจอแล้วจะบอกว่ามีคิวกับแพทย์คนไหนอยู่ กฎนี้เช็กในโปรแกรม เพราะต้องดูวันเวลาจาก schedules ร่วมด้วย

ข้อ 25 ทำไมช่องของวันนี้ที่เลยเวลาแล้วไม่แสดง?

จองเวลาที่ผ่านไปแล้วไม่มีความหมาย SQL จึงกรองด้วย slot_time > strftime('%H:%M', 'now', 'localtime') และตอนกดบันทึกเช็กซ้ำอีกครั้ง กฎนี้ใส่เป็น CHECK ไม่ได้ เพราะ SQLite ไม่ยอมให้ใช้ date('now') ใน CHECK (ทดสอบแล้วได้ "non-deterministic use of date() in a CHECK constraint")

D1 · ค้นหาคิว ยกเลิก และบันทึกว่าตรวจแล้ว

ข้อ 26 ค้นหาหลายเงื่อนไขได้อย่างไรโดยไม่ต่อสตริง?

เขียนเงื่อนไขทุกช่องไว้ในคำสั่งเดียวแบบ (? = '' OR p.phone = ?) ถ้าช่องว่าง ส่วนแรกเป็นจริง เงื่อนไขนั้นจึงไม่มีผล กรอกหลายช่องต้องตรงทุกช่อง (AND) ชื่อใช้ LIKE กับ '%' + คำค้น + '%'

ข้อ 27 ถ้ากดยกเลิกคิวที่ยกเลิกไปแล้ว หรือคิวที่ตรวจแล้ว จะเกิดอะไร?

โปรแกรมอ่านสถานะล่าสุดจากฐานข้อมูลก่อน ถ้าไม่ใช่ "จองแล้ว" จะเตือนและไม่ทำต่อ คำสั่ง UPDATE ยังมีเงื่อนไข AND status = 'จองแล้ว' ซ้อนไว้อีกชั้น ถ้าสถานะถูกเปลี่ยนไปก่อน คำสั่งจะแก้ 0 แถว

ข้อ 28 ทำไมบันทึกว่าตรวจแล้วกับคิวของพรุ่งนี้ไม่ได้?

ผู้ป่วยยังไม่ได้มาตรวจ ถ้ายอมให้บันทึก ข้อมูลจะไม่ตรงความจริง จึงบันทึกได้เฉพาะคิวที่วันนัดเป็นวันนี้หรือผ่านมาแล้ว และสถานะต้องเป็น "จองแล้ว"

ข้อ 29 ทำไมตอนบันทึกว่าตรวจแล้วต้องกรอกผลวินิจฉัย แต่การรักษาไม่บังคับ?

กฎของระบบคือทุกคิวที่ตรวจแล้วต้องมีผลการรักษา และผลวินิจฉัยคือข้อมูลขั้นต่ำของการตรวจ จึงบังคับทั้งในฟอร์ม (เตือนถ้าว่าง) และในฐานข้อมูล (diagnosis NOT NULL)

บางครั้งตรวจแล้วไม่ต้องรักษาต่อ เช่น ตรวจภายในประจำปีผลปกติ ช่องการรักษาจึงว่างได้ เก็บเป็น NULL และแสดงเป็น - ในประวัติ

ข้อ 30 ถ้ากดยกเลิกในฟอร์มผลวินิจฉัย หรือบันทึกไม่สำเร็จ สถานะคิวจะเป็นอย่างไร?

กดยกเลิกหรือ Esc ในฟอร์ม » ask_treatment() คืน None โปรแกรมหยุดก่อนรัน SQL คิวยังเป็น "จองแล้ว"

ถ้าบันทึกแล้วฐานข้อมูลปฏิเสธ transaction จะ rollback ทั้ง UPDATE และ INSERT คิวยังเป็น "จองแล้ว" และไม่มีแถวใน treatments แล้วแสดงข้อความจาก error_to_thai()

ข้อ 31 ทำไมตัวเลขบนแถบหัวเปลี่ยนทันทีหลังบันทึก?

ท้าย mark_done(), cancel_appointment() และ save_booking() เรียก header_bar.refresh() ซึ่งรัน SQL สรุปวันนี้ใหม่ ตัวอย่าง: บันทึกคิว 7 แล้วเปลี่ยนจาก 7 / 0 / 6 / 1 เป็น คิวทั้งหมด 7 · ตรวจแล้ว 1 · รอตรวจ 5 · ยกเลิก 1 ส่วนการจองหรือยกเลิกคิวของพรุ่งนี้ ตัวเลขวันนี้ไม่เปลี่ยน

ข้อ 32 ตอนเลือกแถวในตาราง ทำไมโปรแกรมไปดึงข้อมูลจากฐานข้อมูลใหม่ ไม่อ่านจากตารางบนหน้าจอ?

ค่าที่อ่านจาก Treeview ถูกแปลงเป็นตัวเลขอัตโนมัติ เบอร์ 0812345678 จะกลายเป็น 812345678 เลข 0 หาย เราจึงตั้ง iid ของแถวเป็นรหัส แล้วใช้รหัสนั้นดึงข้อมูลล่าสุดจากฐานข้อมูล ได้ทั้งค่าที่ถูกต้องและสถานะล่าสุด